UNIVERSITÀ POLITECNICA DELLE MARCHE

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Corso di Laurea Magistrale in Ingegneria Informatica e dell'Automazione

Insegnamento di New Generation Databases

ANOMALY DETECTION SU DATI BIOMETRICI

RELAZIONE DI PROGETTO

Docente:

Prof.ssa Claudia Diamantini

Candidati:

Andrea Altieri

Andrea Flaiani

Anno Accademico 2026/2027

Indice

1 Introduzione e dominio applicativo **1**

1.1 Contesto e motivazioni del progetto . . . 1
1.2 Il dominio clinico e la scelta del dataset VitalDB . . . 1
1.3 Le scelte architetturali in sintesi e struttura della relazione . . . 2

2 Architettura della data pipeline **4**

2.1 Il paradigma della Medallion Architecture . . . 4
2.2 Layer Bronze: ingestione e storicizzazione dei dati grezzi . . . 5
2.3 Layer Silver: bonifica, arricchimento e partizionamento . . . 6
2.4 Evoluzione del modello dati: da tabella Parquet a documento BSON . . . 9

3 Archiviazione e governance con MongoDB Time Series **11**

3.1 Layer Gold: perché abbiamo scelto MongoDB Time Series . . . 11
3.2 Benchmark di storage, latenza e sparsità (500 casi) . . . 12
3.3 Data governance: perché un data lakehouse “simulato” . . . 13
3.3.1 Catalogo, macchina a stati e provenienza (collezione registry) . . . 14
3.3.2 Catalogo delle tracce e sincronizzazione automatica (track_metadata) . . . 15
3.3.3 Limiti transazionali delle Time Series, protocollo a due fasi e recovery . . . 15
3.4 Pipeline di aggregazione e ottimizzazione delle query . . . 16
3.4.1 Downsampling lato database con $dateTrunc . . . 16
3.4.2 Eliminazione del problema N+1 sull’elenco dei pazienti (e relativo limite) . . . 17
3.5 Metriche di data quality per sensore e analisi critica . . . 18

4 Modelli di anomaly detection **19**

4.1 Perché un approccio ibrido: regole cliniche + machine learning non supervisionato . . . 19
4.2 Caricamento dal layer Gold e regole cliniche con filtro di freschezza . . . 19
4.2.1 Shock Index clinico . . . 20
4.2.2 Ipotensione severa concomitante a ipossiemia . . . 20
4.3 Ingegnerizzazione delle feature temporali sulla frequenza cardiaca . . . 21
4.4 Modelli di machine learning non supervisionato . . . 21
4.4.1 Imputazione locale per paziente e correzione del bug di data leakage nel `.bfill()` . . . 21
4.4.2 Isolation Forest e significato della soglia `contamination=0.05` . . . 22
4.4.3 LSTM Autoencoder in PyTorch e finestre scorrevoli zero-copy . . . 22
4.5 Valutazione empirica sui 500 casi e interpretazione delle metriche . . . 25
4.6 Persistenza su MongoDB con pattern Insert-Before-Delete e limiti del modulo . . . 26

5 Ingegnerizzazione del sistema, interfaccia grafica e conclusioni **29**

5.1 Backend REST API con FastAPI . . . 29

5.2 Dashboard web e interfaccia utente interattiva 31
5.2.1 Vista di telemetria clinica e anomaly detection interattiva 32
5.2.2 Vista di Data Lakehouse, Governance e Benchmark 34
5.3 Containerizzazione Docker, orchestrazione, test (`pytest`) e reset 35
5.3.1 Orchestrazione della pipeline end-to-end (`scripts/run_pipeline.py`) 36
5.3.2 Suite di 38 test automatizzati (`pytest`) e script di reset 36
5.4 Conclusioni, riepilogo dei limiti per layer e sviluppi futuri 37
5.4.1 Quadro riepilogativo dei limiti riscontrati lungo l'architettura 37
5.4.2 Sviluppi futuri 39

CAPITOLO 1

Introduzione e dominio applicativo

1.1 Contesto e motivazioni del progetto

Durante un intervento chirurgico i monitor multiparametrici di sala operatoria registrano in continuo i parametri vitali del paziente: frequenza cardiaca, saturazione d'ossigeno e pressione arteriosa. Queste serie temporali sono preziose per individuare in anticipo instabilità emodinamiche, ma nella pratica clinica presentano tre problemi concreti per chi deve progettarne la base di dati e l'analisi automatica:

1. **Rumore e artefatti strumentali:** il movimento del paziente, l'uso del bisturi elettrico o il distacco momentaneo di un sensore generano picchi fuori scala (ad esempio frequenze cardiache sopra i 250 bpm o pressioni nulle) e brevi interruzioni di segnale;
2. **Frequenze di campionamento asimmetriche:** mentre frequenza cardiaca e saturazione arrivano ogni secondo (1 Hz), la pressione arteriosa non invasiva (NIBP) viene misurata da un bracciale pneumatico solo ogni 2–5 minuti. Ne deriva una tabella fortemente sparsa, in cui oltre due terzi delle celle grezze sono vuote;
3. **Assenza di etichette cliniche istante per istante:** nei dataset ospedalieri reali raramente esiste un medico che abbia annotato manualmente ogni singolo secondo come "normale" o "anomalo".

L'obiettivo di questo progetto, sviluppato per il corso di *New Generation Databases*, è costruire da zero una piattaforma completa che gestisca l'intero ciclo di vita di questi dati: dall'estrazione dei tracciati grezzi alla pulizia, dall'archiviazione su un database NoSQL per serie temporali (**MongoDB 8.0 Time Series**) al rilevamento delle anomalie e alla visualizzazione su dashboard web.

1.2 Il dominio clinico e la scelta del dataset VitalDB

Per sviluppare e testare la pipeline abbiamo scelto **VitalDB**, un dataset pubblico raccolto dal Seoul National University Hospital che contiene le registrazioni ad alta risoluzione di oltre 6.300 interventi chirurgici.

Perché VitalDB e non MIMIC-IV. In fase iniziale abbiamo valutato anche MIMIC-IV, che è lo standard più citato in letteratura per la terapia intensiva. Lo abbiamo scartato per un motivo pratico: l'accesso a MIMIC-IV su PhysioNet richiede un iter di accreditamento formale e tempi di attesa incompatibili con lo sviluppo del progetto, mentre VitalDB offre un'API REST pubblica e una libreria Python (`vitaldb`) che permettono di scaricare i tracciati in modo programmatico e riproducibile.

Perché solo serie numeriche a 1 Hz e non le forme d'onda. VitalDB mette a disposizione sia le forme d'onda grezze (ECG e pletismografia a 100–500 Hz) sia i parametri numerici già estratti dal monitor a 1 Hz. Abbiamo scelto di lavorare esclusivamente sui parametri numerici a 1 Hz perché il focus del corso è l'architettura dati, la modellazione documentale e la data governance:

processare forme d'onda grezze avrebbe spostato il lavoro sul *digital signal processing* (filtraggio in frequenza, estrazione dei picchi R-R) senza aggiungere valore alla parte di database.

Il nostro studio si concentra su cinque tracce registrate dal monitor `Solar8000`:

- `Solar8000/HR`: frequenza cardiaca in battiti al minuto (bpm), campionata a 1 Hz;
- `Solar8000/PLETH_SPO2`: saturazione periferica dell'ossigeno (%), campionata a 1 Hz;
- `Solar8000/NIBP_SBP`: pressione arteriosa sistolica non invasiva (mmHg), intermittente;
- `Solar8000/NIBP_DBP`: pressione arteriosa diastolica non invasiva (mmHg), intermittente;
- `Solar8000/NIBP_MBP`: pressione arteriosa media (mmHg), intermittente.

Sul piano fisiologico, la pressione arteriosa media ($MBP \approx DBP + \frac{1}{3}(SBP - DBP)$) rappresenta la grandezza principale con cui l'anestesista valuta la perfusione degli organi: le linee guida indicano la soglia di $MBP < 65$ mmHg come limite critico sotto il quale aumenta il rischio di danno ischemico renale e cerebrale.

Accanto ai cinque segnali temporali, la pipeline scarica i metadati clinici dei pazienti (`clinical_data.parquet`, contenente età, sesso e reparto chirurgico `department`) e i referti di laboratorio perioperatori (`lab_data.parquet`). La coorte sperimentale su cui abbiamo eseguito l'intera pipeline comprende **500 interventi chirurgici**, pari a **5.565.384 secondi grezzi** nel livello Bronze e **5.402.386 secondi attivi** nel livello Gold (oltre 1.500 ore di monitoraggio intraoperatorio).

Quadro sintetico della coorte sperimentale

- **Perimetro analizzato:** 500 casi clinici completi ($5.402.386$ campioni a 1 Hz nel layer Gold).
- **Segnali continui (1 Hz):** `Solar8000/HR` (bpm) e `Solar8000/PLETH_SPO2` (%).
- **Segnali intermittenti (ogni 2–5 min):** `NIBP_SBP`, `NIBP_DBP`, `NIBP_MBP` (mmHg).
- **Metadati di contesto:** reparto chirurgico (`department`), età e sesso da `clinical_data.parquet`.

1.3 Le scelte architetturali in sintesi e struttura della relazione

Per rendere chiara fin da subito la logica del progetto, riassumiamo le sei decisioni tecniche principali che abbiamo preso e che approfondiremo nei capitoli successivi:

1. **Pipeline Medallion a tre layer (Bronze $\rightarrow$ Silver $\rightarrow$ Gold):** abbiamo separato il dato grezzo immutabile (Bronze), il dato pulito e arricchito con i flag di qualità (Silver) e il dato servito su database (Gold). In questo modo, quando abbiamo dovuto correggere la logica di pulizia o di imputazione, abbiamo potuto rigenerare Silver e Gold partendo dai file locali senza riscaricare i 500 casi da VitalDB;
2. **Formato Apache Parquet nei layer Bronze e Silver:** invece di salvare i tracciati in CSV, abbiamo usato Parquet perché è un formato colonnare compresso che preserva i tipi numerici a 64 bit, occupa una frazione dello spazio ed è immediato da partizionare in sottocartelle per reparto chirurgico (`department=<REPARTO>`);
3. **MongoDB 8.0 Time Series per il layer Gold:** per servire le query della dashboard e dei modelli abbiamo scelto una collezione Time Series nativa (`vital_signals`). Rispetto a mantenere i dati solo su file Parquet o in una tabella relazionale classica, MongoDB raggruppa i secondi contigui in *bucket* compressi, indicizza i metadati del paziente (`metaField`) e omette fisicamente i campi nulli nei secondi in cui il bracciale pressorio non misura;

4. **Data Lakehouse "simulato" su MongoDB e Python:** strumenti industriali come Delta Lake, Apache Iceberg o Apache Hudi offrono transazioni ACID e cataloghi distribuiti, ma richiedono un cluster Spark o Trino che sarebbe stato sproporzionato per 500 casi su una singola macchina e fuori tema rispetto a MongoDB. Abbiamo quindi ricostruito a livello applicativo le garanzie essenziali — catalogo `registry` con stati `loading`/`committed`/`failed`, tracciabilità tramite `run_id` e `pipeline_version`, e fonte unica dei limiti fisiologici in `TRACK_SPECS` — dichiarando in modo trasparente dove questo approccio si ferma rispetto a un lakehouse nativo;
5. **Anomaly detection ibrida (regole cliniche + ML non supervisionato):** non avendo etichette cliniche verificate, abbiamo combinato due regole mediche note (*Shock Index* > 0.9 e ipotensione severa con vincolo di freschezza pressoria ≤ 120 s) con due modelli non supervisionati (*Isolation Forest* e *LSTM Autoencoder* in PyTorch), usando le regole anche come riferimento (*pseudo-ground truth*) per misurare la concordanza dei modelli;
6. **Stack FastAPI + Dashboard Vanilla JS + Docker Compose:** abbiamo esposto il layer Gold tramite un backend FastAPI e un'interfaccia web scritta in HTML5, CSS3 e JavaScript puro (senza framework di build come React o Webpack), impacchettando database e applicazione in due container Docker orchestrati con *healthcheck*.

La relazione segue l'ordine del flusso dei dati: il **Capitolo 2** descrive i layer Bronze e Silver e il passaggio da tabella Parquet a documento BSON; il **Capitolo 3** analizza il layer Gold su MongoDB Time Series, i risultati del benchmark e l'architettura di data governance; il **Capitolo 4** tratta le regole cliniche, i modelli di machine learning e la loro valutazione; il **Capitolo 5** presenta infine il backend FastAPI, la dashboard, la suite di test automatici e un bilancio complessivo dei limiti di ciascun componente.

CAPITOLO 2

Architettura della data pipeline

2.1 Il paradigma della Medallion Architecture

Per gestire oltre cinque milioni di campioni biometrici senza confondere i dati grezzi con quelli trasformati, abbiamo organizzato la pipeline secondo la **Medallion Architecture**, un pattern tipico dei data lakehouse che suddivide il flusso in tre livelli: **Bronze**, **Silver** e **Gold**.

Come mostrato nella Figura 2.1, ogni livello ha una responsabilità precisa:

- **Layer Bronze (dati grezzi immutabili):** salva su disco locale i tracciati esattamente come vengono restituiti da VitalDB, senza modificare né filtrare alcun valore;
- **Layer Silver (pulizia, arricchimento e partizionamento):** individua gli outlier fisiologici, imputa i buchi brevi e le pressioni intermittenti, aggiunge i flag di qualità e partiziona i file per reparto chirurgico;
- **Layer Gold (archiviazione Time Series e serving):** rimodella le righe in documenti BSON e li carica nella collezione temporale di MongoDB 8.0 per alimentare le query dell'API, la dashboard e i modelli di anomaly detection.



Figura 2.1: Panoramica dell'architettura Medallion e dei livelli Bronze, Silver e Gold generata in TikZ.

Perché abbiamo scelto di separare i tre layer. L'alternativa più semplice sarebbe stata scaricare i dati da VitalDB, pulirli al volo in memoria e inserirli direttamente in MongoDB. Abbiamo scartato questa scorciatoia perché il download di 500 casi dall'API remota di VitalDB richiede tempo ed è soggetto a errori di rete: avere il layer Bronze salvato su disco ci ha permesso,

durante lo sviluppo, di correggere più volte le regole di pulizia e di imputazione del layer Silver rigenerando l'intera pipeline in pochi minuti senza mai riscaricare i dati dalla rete.

L'intera catena operativa, dall'estrazione remota fino alla visualizzazione web, è riassunta nelle cinque fasi della Figura 2.2.

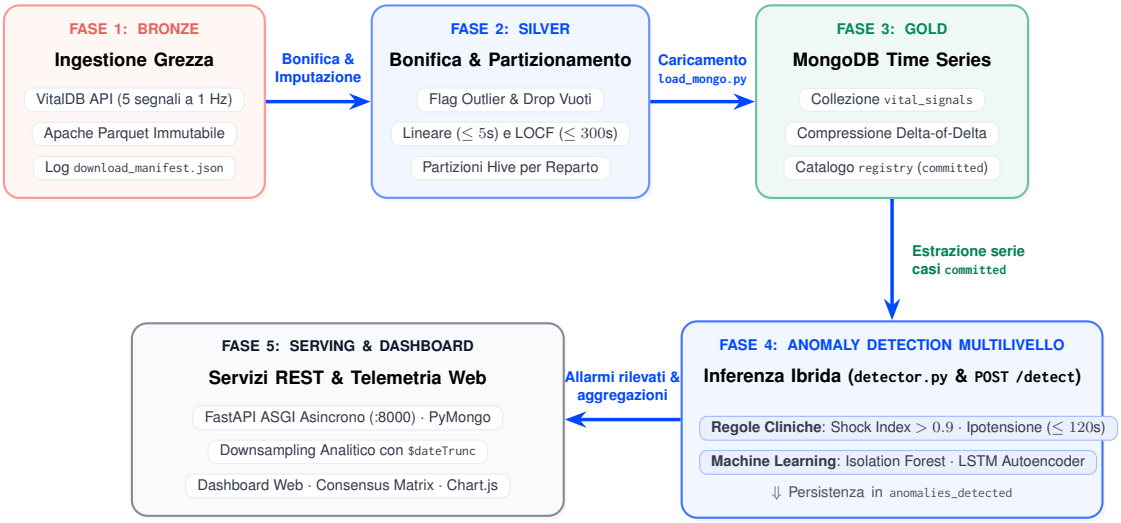


Figura 2.2: Architettura della pipeline di esecuzione end-to-end dall'ingestione alla visualizzazione.

2.2 Layer Bronze: ingestione e storicizzazione dei dati grezzi

Il livello Bronze è implementato nello script `src/bronze/download.py`. La funzione `find_available_cases` interroga l'archivio VitalDB tramite `vitaldb.find_cases(tracks)` per trovare gli interventi che contengono tutte e cinque le tracce di interesse (`HR`, `PLETH_SPO2`, `NIBP_SBP`, `NIBP_DBP`, `NIBP_MBP`). Di questi viene selezionato il perimetro configurato (`MAX_CASES = 500` in `src/config.py`) e, per ciascun paziente, la funzione `download_case` scarica la matrice temporale a passo di un secondo (`SAMPLING_INTERVAL = 1.0`), aggiungendo la colonna `Time` e salvando il risultato in `data/bronze/case_<ID>.parquet`.

Perché abbiamo usato Apache Parquet invece di CSV. Salvare $5,56$ milioni di righe distribuite su 500 file CSV avrebbe comportato tre svantaggi: file molto più pesanti su disco, parsing testuale lento ad ogni lettura e necessità di ridedurre i tipi di dato (`float64`) ad ogni apertura. Con **Apache Parquet** i dati sono salvati in formato binario colonnare compresso (Snappy): l'intero layer Bronze di 500 casi occupa appena **33,95 MB** e viene riletto da `pandas.read_parquet` in pochi millisecondi mantenendo i tipi numerici esatti.

Gestione dei dati clinici (fail-fast) e dei dati di laboratorio (graceful fallback). Oltre ai tracciati a 1 Hz, il modulo Bronze scarica due tabelle di contesto:

1. **Metadati clinici (`clinical_data.parquet`):** contengono età, sesso e reparto chirurgico (`department`). Sviluppando lo script ci siamo accorti che la funzione ufficiale `vitaldb.load_clinical_data()`, se chiamata senza argomenti, restituisce una tabella vuota a causa di un bug della libreria. Per risolvere il problema, `download_clinical_data` interroga direttamente l'endpoint REST `https://api.vitaldb.net/cases` e usa l'SDK parametrizzato (`caseids=list(range(1, 6389))`) come fallback. Se entrambi i tentativi falliscono, lo script interrompe subito la pipeline con `sys.exit(1)` (**politica fail-fast**): senza la colonna `department`, infatti, il layer Silver non saprebbe come partizionare i casi e li metterebbe tutti nella cartella `department=UNKNOWN`, nascondendo l'errore;

2. **Referti di laboratorio (`lab_data.parquet`):** la funzione `download_lab_data` scarica da `https://api.vitaldb.net` (con fallback su `vitaldb.load_lab_data`) le analisi ematochimiche filtrate sui soli `case_id` della coorte. A differenza dei dati clinici, qui abbiamo scelto una politica non bloccante (**graceful fallback**): se l'endpoint non risponde, il manifest registra `"status": "unavailable"` ma la pipeline prosegue, poiché i dati di laboratorio sono salvati nel Bronze per completezza ma non vengono usati dai layer successivi.

Manifest di esecuzione e controllo sui file esistenti. Ad ogni avvio di `download.py` viene generato un nuovo identificativo di esecuzione tramite `config.get_or_create_run_id(force_new=True)` (salvato anche nel file `data/.current_run_id`) e viene scritto il file `data/download_manifest.json`. Il manifest registra `run_id`, `pipeline_version` (`"1.1.0"`), `max_cases_configured`, `sampling_interval`, la lista `tracks` e, per ciascuno dei 500 casi, lo stato (`downloaded`, `skipped_existing` o `failed`), il numero di righe e i byte su disco.

Per evitare di riscaricare file già presenti quando si rilancia lo script, `download_case` controlla se `case_<ID>.parquet` esiste già. Prima di marcarlo come `skipped_existing`, però, apre il file e verifica che contenga tutte le tracce richieste più la colonna `Time` e che la differenza tra i primi due istanti (`Time.iloc[1] - Time.iloc[0]`) coincida con `SAMPLING_INTERVAL` ($1,0$ s).

Limiti del layer Bronze

- **Verifica leggera sui file già presenti:** il controllo prima di `skipped_existing` verifica i nomi delle colonne e il passo tra i primi due campioni, ma non calcola un hash crittografico del file né controlla che il Parquet provenga dallo stesso `run_id`. È un compromesso scelto per non rallentare l'avvio su 500 file.
- **Tabella `lab_data.parquet` non integrata a valle:** i referti di laboratorio vengono scaricati e salvati nel Bronze, ma non sono ancora uniti alle serie a 1 Hz nel Silver o nel Gold.

2.3 Layer Silver: bonifica, arricchimento e partizionamento

I file Parquet del layer Bronze contengono il $66,3\%$ di celle vuote (`NaN`), oltre a valori fisiologicamente impossibili dovuti a disturbi sui sensori. Il layer Silver, implementato in `src/silver/clean.py`, trasforma ciascun caso applicando una sequenza precisa di operazioni progettata per pulire il segnale senza inventare dati artificiali né perdere informazioni utili per l'audit.

1. Riallineamento del perimetro dei casi. All'avvio, `process_all_cases` legge `download_manifest.json` (se generato con lo stesso `MAX_CASES`) per sapere esattamente quali `case_id` fanno parte del run corrente, e cancella da `data/silver/` eventuali file Parquet rimasti da esecuzioni precedenti con un numero diverso di casi. Abbiamo aggiunto questa pulizia automatica dopo esserci accorti che, cambiando `MAX_CASES` in `config.py`, nella cartella Silver potevano restare file "orfani" di run più grandi che falsavano i conteggi del report di qualità.

2. Mantenimento della griglia temporale a 1 Hz fino a fine pulizia. Un errore facile da commettere con `pandas` è chiamare `dropna(how='all')` all'inizio della pulizia. Se lo facessimo, due campioni separati da dieci minuti di sensore staccato diventerebbero righe consecutive nel DataFrame: a quel punto un'interpolazione con `limit=5` o un forward-fill con `limit=300` conterebbero il numero di righe anziché i secondi reali. Per questo motivo in `clean_case` manteniamo tutte le righe a 1 Hz durante l'intera fase di controllo e imputazione, ed eliminiamo le righe in cui tutti e cinque i parametri vitali sono contemporaneamente nulli (`dropna(subset=vital_cols, how='all')`) **solo all'ultimo passaggio**. Sui 500 casi, questa scelta porta a scartare appena **162.998 righe**

su 5.565.384 (**2,93%**), che corrispondono quasi solo ai minuti iniziali e finali in cui il monitor è acceso ma il paziente non è ancora collegato.

3. Fonte unica dei range fisiologici (`TRACK_SPECS`) e outlier calcolati PRIMA dell'imputazione. I limiti minimi e massimi fisiologici di ciascun parametro sono definiti in un unico dizionario centralizzato, `TRACK_SPECS` in `src/config.py` (Tabella 2.1). All'inizio del progetto questi range erano scritti sia in `clean.py` sia nello script MongoDB `mongo/registry_schema.js`, e avevamo finito per avere due soglie diverse per la pressione diastolica (180 mmHg nel codice Python contro 200 mmHg nel database); centralizzarli in `config.py` ha eliminato alla radice queste incongruenze.

Traccia VitalDB	Flag outlier / imputazione	Range valido (`TRACK_SPECS`)	Unità
`Solar8000/HR`	`HR_outlier`, `HR_imputed`	$20 – 250$	bpm
`Solar8000/PLETH_SPO2`	`SPO2_outlier`, `SPO2_imputed`	$50 – 100$	%
`Solar8000/NIBP_SBP`	`SBP_outlier`, `SBP_imputed`	$40 – 250$	mmHg
`Solar8000/NIBP_DBP`	`DBP_outlier`, `DBP_imputed`	$10 – 180$	mmHg
`Solar8000/NIBP_MBP`	`MBP_outlier`, `MBP_imputed`	$20 – 220$	mmHg

Tabella 2.1: Limiti fisiologici definiti in `src/config.py` (`TRACK_SPECS`) e relativi flag nel layer Silver.

Quando un valore osservato cade fuori dal range di Tabella 2.1, **non lo cancelliamo**: lasciamo il numero originale nella cella e impostiamo a `True` il relativo flag booleano `*_outlier`. La scelta cruciale è che calcoliamo i flag `*_outlier` **sui dati grezzi prima dell'interpolazione e del forward-fill**, e durante l'imputazione mascheriamo temporaneamente a `NaN` i punti outlier (`raw_col.mask(out_mask)`). Se invece calcolassimo gli outlier dopo il forward-fill — come facevamo nella prima versione del codice — una singola lettura errata del bracciale (es. SBP $= 10$ mmHg) verrebbe copiata in avanti per 300 secondi e contata come 301 outlier consecutivi.

4. Imputazione differenziata: buchi brevi (≤ 5 s) per HR/SpO$_2$ e LOCF (≤ 300 s) per NIBP. Trattiamo in modo diverso i segnali continui e quelli intermittenti:

- **Segnali continui (`HR` e `PLETH_SPO2`):** usiamo la funzione `interpolate_short_gaps(series, max_gap=5)`. Non abbiamo usato il semplice `.interpolate(limit=5)` di `pandas` perché quest'ultimo riempie i primi 5 secondi anche quando il sensore si stacca per dieci minuti, creando una "coda" artificiale all'inizio di ogni disconnessione. La nostra funzione calcola invece la lunghezza di ogni blocco di `NaN` e interpola linearmente **solo i buchi interni lunghi al massimo 5 secondi** compresi tra due misure valide, segnando i punti ricostruiti in `HR_imputed` e `SPO2_imputed`;
- **Segnali pressori intermittenti (`NIBP_SBP`, `NIBP_DBP`, `NIBP_MBP`):** poiché il bracciale misura ogni 2–5 minuti, applichiamo il *Last Observation Carried Forward* (`ffill(limit=300)`) mantenendo valido l'ultimo valore non-outlier per un massimo di 300 secondi (5 minuti). Oltre i 5 minuti il valore torna a `NaN` perché in chirurgia una pressione più vecchia di cinque minuti non è più attendibile. Ogni canale ha il proprio flag booleano esplicito (`SBP_imputed`, `DBP_imputed`, `MBP_imputed`), così da contare con precisione le celle riempite su ciascuna componente;
- **Età della misura pressoria (`nibp_age_s`):** per sapere sempre da quanti secondi non viene gonfiato il bracciale, calcoliamo in ogni riga `nibp_age_s` $= t - t_{\text{last_cuff}}$. Nei secondi iniziali dell'intervento che precedono la prima lettura NIBP valida, la differenza è indefinita e viene riempita con il valore sentinella `9999.0`, così che a valle venga riconosciuta subito come misura non fresca.

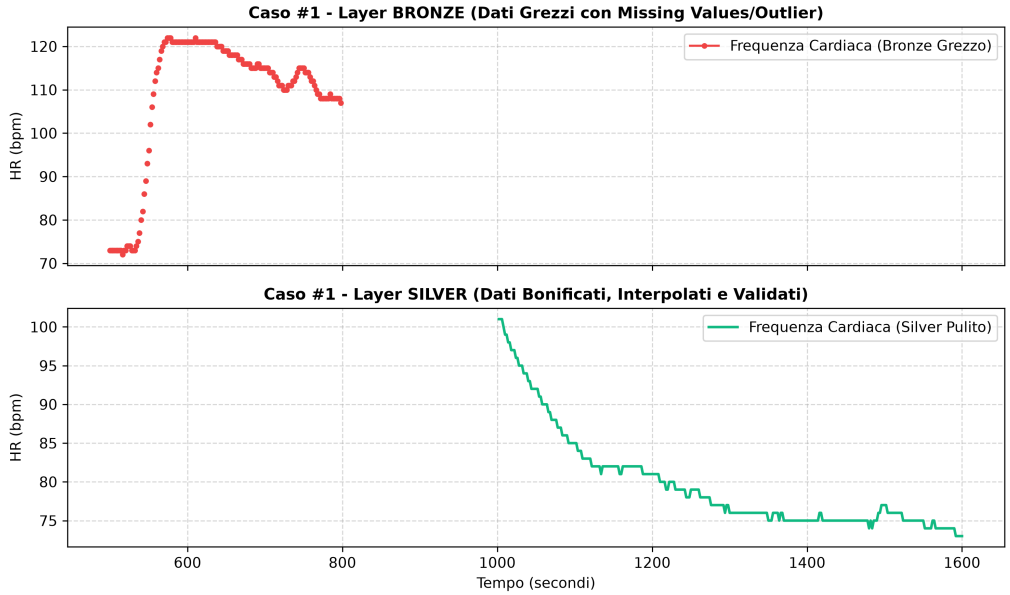


Figura 2.3: Confronto visivo tra il tracciato della frequenza cardiaca grezzo (Bronze) e bonificato (Silver).

La Figura 2.3 mostra l'effetto di questa pulizia su un tracciato reale di frequenza cardiaca, mentre il Listato 2.1 riporta i passaggi principali di `clean_case` in `src/silver/clean.py`.

```
// Codice estratto da src/silver/clean.py
# 1. Flag outlier calcolati SUI DATI GREZZI (da config.TRACK_SPECS) prima di imputare
for track_name, flag_col in outlier_map.items():
    spec = track_specs.get(track_name, {})
    val = df_clean[track_name]
    df_clean[flag_col] = val.notna() & ((val < spec["min"]) | (val > spec["max"]))

# 2. Segnali continui (HR e SpO2): interpola solo buchi interni <= 5s senza propagare outlier
clean_hr, hr_imp = interpolate_short_gaps(raw_hr.mask(out_hr), max_gap=5)
df_clean['Solar8000/HR'] = clean_hr.where(~out_hr, raw_hr)
df_clean['HR_imputed'] = hr_imp & (~out_hr)

# 3. Segnali NIBP: calcolo nibp_age_s (sentinella 9999.0) e LOCF max 300s per traccia
last_measure_time = time_series.where(any_valid_nibp).ffill()
df_clean['nibp_age_s'] = (time_series - last_measure_time).fillna(9999.0).astype(float)
for t, o, imp_col in present_nibp:
    raw_col = df_clean[t]
    out_mask = df_clean[o]
    was_na = raw_col.isna()
    filled_col = raw_col.mask(out_mask).ffill(limit=300)
    df_clean[t] = filled_col.where(~out_mask, raw_col)
    df_clean[imp_col] = was_na & df_clean[t].notna()

```

continua dalla pagina precedente...

```
# 4. Scarta solo alla fine le righe con tutte e 5 le tracce vitali
    contemporaneamente nulle
df_clean = df_clean.dropna(subset=vital_cols, how='all').copy()
```

Listato 2.1: Identificazione preventiva degli outlier, interpolazione dei gap brevi e LOCF nel layer Silver.

5. Partizionamento fisico per reparto e report di qualità. I DataFrame puliti vengono salvati in sottocartelle nominate secondo la convenzione Hive:

`data/silver/department=<REPARTO>/case_<ID>.parquet`

Per associare ogni `case_id` al reparto leggiamo `clinical_data.parquet` (convertendo le chiavi da `numpy.int64` a `int` nativo Python per evitare mancati match nel dizionario). Abbiamo scelto di partizionare per reparto chirurgico perché è il criterio di filtro più naturale quando si vogliono analizzare sottogruppi di interventi (es. *General surgery* o *Thoracic surgery*) leggendo da disco solo la cartella interessata (*partition pruning*). Al termine del ciclo, lo script salva il report completo `data/quality_report.json` (archiviando la versione precedente in `data/quality_history/`).

Limiti del layer Silver

- **Assenza di un quality gate bloccante a livello di caso:** `clean.py` marca ogni singolo secondo anomalo o imputato, ma non scarta un intero paziente se ha una copertura molto bassa (ad esempio se il sensore HR è rimasto staccato per metà intervento). Un caso povero di dati viene comunque salvato in Silver e caricato in Gold, purché abbia almeno una riga non vuota.
- **Approssimazione del forward-fill (LOCF):** mantenere costante la pressione per un massimo di 300 s riduce i `NaN` dal $66,3\%$ al $26,3\%$, ma resta una semplificazione a gradini: variazioni pressorie rapide che avvengono tra due gonfiaggi del bracciale non sono osservabili.

2.4 Evoluzione del modello dati: da tabella Parquet a documento BSON

Passando da Bronze a Silver e infine a Gold cambia anche la forma del dato:

- nel **Bronze** ogni file Parquet ha **6 colonne** (`Time` più le 5 tracce vitali);
- nel **Silver** la tabella sale a **18 colonne**: alle 6 colonne base si aggiungono `case_id`, i 5 flag `*_outlier`, i 5 flag `*_imputed` e il contatore `nibp_age_s`. Proprio l'aggiunta di queste 12 colonne di governance e il riempimento dei `NaN` tramite LOCF spiegano perché su disco il layer Silver occupi più spazio del Bronze (**44,52 MB** contro **33,95 MB**, cioè **+31,1%**);
- nel **Gold** su MongoDB ogni secondo attivo viene trasformato da `build_records` (in `src/gold/load_mongo.py`) in un documento gerarchico BSON inserito nella collezione `vital_signals`, che comprime il tutto a **15,60 MB** di storage puro.

Come mostrato nel Listato 2.2, il documento BSON separa nettamente quattro blocchi:

1. `timestamp`: istante temporale convertito in `ISODate` sommando i secondi relativi `Time` alla data base convenzionale `2020-01-01T00:00:00Z`;
2. `metadata`: il sotto-documento immutabile per paziente (`case_id`, `sensor_name`, `department`, `age`, `sex`) usato da MongoDB come `metaField` per raggruppare e indicizzare i bucket;

3. `metrics`: contiene **solo** i parametri vitali non nulli in quel secondo (con i caratteri `/` sostituiti da _ nelle chiavi). Se in quel secondo una misura manca, la chiave non viene scritta nel dizionario, occupando zero byte;
4. `quality_flags`: raccoglie i 5 flag `*_outlier`, i 5 flag `*_imputed` e `nibp_age_s`, così i moduli di anomaly detection possono filtrare gli artefatti e controllare la freschezza pressoria direttamente dal layer Gold.

```
// Documento BSON archiviato nella collezione 'vital_signals'
{
  "_id": ObjectId("65d3f1a0e4b01a2b3c4d5e6f"),
  "timestamp": ISODate("2020-01-01T08:15:30Z"),
  "metadata": {
    "case_id": 1,
    "sensor_name": "Solar8000",
    "department": "General surgery",
    "age": 64,
    "sex": "M"
  },
  "metrics": {
    "Solar8000_HR": 72.0,
    "Solar8000_PLETH_SPO2": 98.0,
    "Solar8000_NIBP_SBP": 120.0,
    "Solar8000_NIBP_DBP": 80.0,
    "Solar8000_NIBP_MBP": 93.3
  },
  "quality_flags": {
    "HR_outlier": false, "SPO2_outlier": false,
    "SBP_outlier": false, "DBP_outlier": false, "MBP_outlier": false,
    "HR_imputed": false, "SPO2_imputed": false,
    "SBP_imputed": true, "DBP_imputed": true, "MBP_imputed": true,
    "nibp_age_s": 14.0
  }
}
```

Listato 2.2: Struttura del documento BSON inserito nella collezione Time Series vital_signals (layer Gold).

CAPITOLO 3

Archiviazione e governance con MongoDB Time Series

3.1 Layer Gold: perché abbiamo scelto MongoDB Time Series

Il livello **Gold** è lo strato pensato per servire in modo rapido le query della dashboard web, le API REST e il caricamento dei dati nei modelli di machine learning. I file Parquet puliti del livello Silver vengono caricati su un'istanza **MongoDB 8.0** all'interno della collezione Time Series nativa `vital_signals` tramite lo script `src/gold/load_mongo.py`.

Alternative valutate e motivazione della scelta. Per il livello Gold avevamo tre strade possibili:

1. **Restare solo su file Parquet:** ottimo per analisi batch sequenziali in Python, ma scomodo per un'applicazione web che deve filtrare singoli pazienti, aggregare intervalli temporali al volo (downsampling) e salvare contemporaneamente le anomalie rilevate;
2. **Usare una collezione MongoDB tradizionale (o una tabella SQL):** salvare $5,4$ milioni di secondi come $5,4$ milioni di documenti BSON indipendenti avrebbe generato un forte spreco di spazio e di indici, perché ogni singolo secondo avrebbe avuto il proprio `_id`, le stesse chiavi ripetute e una voce nell'indice B-Tree;
3. **Usare una Time Series Collection di MongoDB 8.0 (scelta adottata):** dichiarando la collezione come serie temporale, MongoDB non salva un documento fisico per ogni secondo, ma raggruppa automaticamente i campioni contigui dello stesso paziente dentro blocchi compressi chiamati **bucket** (nella collezione interna `system.buckets.vital_signals`).

La creazione di `vital_signals` in `ensure_timeseries_collections(db)` poggia su tre parametri:

- `timeField: "timestamp"`, che indica a MongoDB il campo temporale su cui ordinare le misure;
- `metaField: "metadata"`, che contiene i dati fissi del paziente (`case_id`, `sensor_name`, `department`, `age`, `sex`) e permette a MongoDB di mettere nello stesso bucket solo misure appartenenti allo stesso intervento;
- `granularity: "seconds"`, che informa il motore che i dati arrivano ogni secondo, calibrando l'ampiezza dei bucket (fino a 1.000 campioni o un'ora di dati per bucket).

All'interno di ciascun bucket, MongoDB comprime le serie in modo colonnare:

- **Compressione dei timestamp (Delta-of-Delta):** poiché i nostri campioni sono distanziati di 1 s costante ($D_i = t_i - t_{i-1} = 1$ s), la differenza seconda $\Delta_i = D_i - D_{i-1}$ vale quasi sempre zero e viene codificata con pochissimi bit invece di 8 byte per ogni secondo;
- **Compressione dei valori numerici (XOR + Zstandard):** parametri come frequenza cardiaca e saturazione variano lentamente da un secondo all'altro, quindi la codifica differenziale seguita da Zstandard riduce fortemente il peso del blocco;
- **Omissione nativa dei campi nulli:** nei file Parquet ogni colonna deve comunque allocare i bit di definizione (*definition levels*) per indicare se una cella è piena o vuota. In BSON, invece, quando in un secondo la pressione NIBP è `NaN`, `build_records` semplicemente non inserisce quella chiave dentro `metrics`, occupando zero byte fisici.

3.2 Benchmark di storage, latenza e sparsità (500 casi)

Per misurare in modo oggettivo cosa cambia passando da Bronze a Silver e a Gold, abbiamo scritto lo script `src/analysis/benchmark_etl.py`, che salva i risultati in `data/benchmark_report.json` marcandoli con lo stesso `run_id` e la stessa `pipeline_version` (`"1.1.0"`) del resto della pipeline. Per garantire un confronto ad armi pari, lo script legge prima dalla collezione `registry` la lista dei **500 casi con stato** `committed` ($5.402.386$ record Gold) e calcola lo spazio occupato su disco filtrando esattamente gli stessi 500 file nei layer Bronze e Silver. I risultati sono riassunti nella Tabella 3.1 e nella Figura 3.1.

Parametro / Metrica	Bronze (Parquet, 6 col.)	Silver (Parquet, 18 col.)	Gold (MongoDB 8.0 TS)
Occupazione disco (500 casi)	33,95 MB	44,52 MB (+31,1% con flag DQ)	**15,60 MB** (-54,1% vs Bronze / -65,0% vs Silver)
Indici B-Tree (`totalIndexSize`)	N/A (File su disco)	N/A (File su disco)	+2,56 MB (Totale `collStats`: 18,16 MB)
Latenza query singolo caso	26,95 ms (Parquet)	26,95 ms (Parquet)	**12,02 ms** (Pushdown su `metaField`)
Fattore di velocizzazione	1,0× (Riferimento)	1,0×	**2,2× più veloce**
Valori mancanti (Null Rate)	66,3%	26,3% (post-LOCF)	26,3% logico / **0 byte fisici** (Omissione BSON)

Tabella 3.1: Confronto di storage, latenza e valori mancanti sui 500 casi clinici (5.402.386 record Gold).

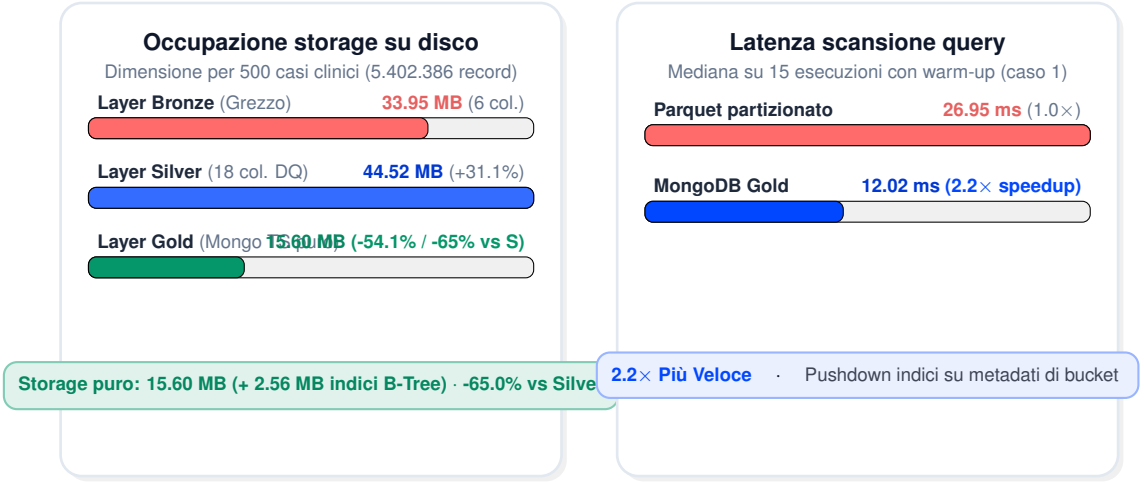


Figura 3.1: Benchmark comparativo dell'occupazione disco e della latenza di query sui 500 casi clinici generato in TikZ.

Dai numeri della Tabella 3.1 emergono tre aspetti importanti che vale la pena spiegare nel dettaglio:

1. **Perché Silver è più grande di Bronze (+31,1%), mentre Gold comprime (-65,0%):** a prima vista potrebbe sorprendere che il layer Silver pulito pesi **44,52 MB** contro i **33,95 MB** del Bronze grezzo. La ragione è che nel nostro progetto il layer Silver non ha lo scopo di comprimere, ma di **arricchire e densificare**: passa da 6 a 18 colonne (aggiungendo 10 flag booleani, `case_id` e `nibp_age_s`) e riempie i buchi pressori con il forward-fill (portando i `NaN` dal $66,3\%$ al $26,3\%$). In particolare, la colonna `nibp_age_s` è un `float64` che cresce di 1 a ogni secondo $(0, 1, 2, \ldots, 300)$, quindi non ha mai valori uguali consecutivi e impedisce a Parquet di usare il *Run-Length Encoding*. Quando invece passiamo al layer Gold su MongoDB Time Series, i bucket compressi scendono a **15,60 MB** di `storageSize` puro (**-65,0%** rispetto a Silver e **-54,1%** rispetto a Bronze). Per onestà abbiamo separato nel report i **15,60 MB** di dati compressi (`storageSize`) dai **2,56 MB** occupati dagli indici B-Tree (`totalIndexSize`, per un totale `collStats` di **18,16 MB**);
2. **Come abbiamo calcolato il Null Rate reale del 26,3% in Gold:** in una prima versione dello script avevamo messo il Null Rate di MongoDB fisso a $0,0\%$ perché nei documenti BSON i campi nulli non esistono fisicamente. Ci siamo però resi conto che scri-

vere $0,0\%$ faceva sembrare che in Gold avessimo recuperato tutte le misure mancanti. Abbiamo quindi modificato `benchmark_etl.py` (righe 88–110) per contare tramite query `{"metrics.<traccia>": {"$exists": True}}` quante volte ciascuna delle 5 tracce è realmente presente rispetto al totale teorico (documenti $\times$ 5). Il risultato è **26,3%**, identico a Silver: questo dimostra che il passaggio a Gold non inventa né perde alcuna misura, pur occupando zero byte su disco per quel $26,3\%$ di valori assenti;

3. **Velocizzazione 2,2$\times$ sulla query per singolo paziente:** per confrontare i tempi di lettura a parità di compito (*like-for-like*), lo script esegue prima un giro di *warmup* e poi misura la mediana su 15 esecuzioni della stessa operazione: calcolare la media di `Solar8000/HR` per il caso `case_id = 1`. Leggere e aggregare il file Parquet partizionato richiede in mediana **26,95 ms**, mentre la pipeline MongoDB (`$match` su `metadata.case_id` seguito da `$group`) impiega **12,02 ms** (**2,2$\times$ più veloce**), sfruttando l'indice sul `metaField` per leggere solo i bucket di quel paziente.

Limiti metodologici del benchmark

Il test di latenza misura un caso d'uso reale della nostra dashboard (interrogazione e aggregazione su un singolo paziente), ma va letto conoscendone i confini: la prova è eseguita su una singola macchina locale con dataset caldo in cache dopo il *warmup* e valuta una query selettiva per `case_id`. Su scansioni analitiche massive dell'intera coorte senza filtri (tutte le $5,4$ milioni di righe insieme), la lettura colonnare diretta dei file Parquet da parte di `pandas` resta competitiva rispetto al passaggio attraverso il protocollo di rete di MongoDB.

3.3 Data governance: perché un data lakehouse "simulato"

Uno dei temi centrali concordati con la docente riguarda la **data governance**: come garantire che i dati caricati in Gold siano completi, validati e ricollegabili ai file Bronze e Silver da cui provengono.

Perché non abbiamo usato Delta Lake, Apache Iceberg o Apache Hudi. Nel mondo industriale un data lakehouse si costruisce con formati tabellari aperti come **Delta Lake**, **Apache Iceberg** o **Apache Hudi**, che offrono transazioni ACID su file Parquet, *schema enforcement* e *time travel* tramite un transaction log JSON/Avro. Nel nostro progetto abbiamo scelto di **non** usare questi framework per due motivi concreti:

1. **Complessità infrastrutturale sproporzionata:** Delta Lake e Iceberg sono progettati per girare sopra motori distribuiti come Apache Spark o Trino. Mettere in piedi un cluster Spark + JVM dentro Docker per gestire 500 casi (44 MB di file Parquet) avrebbe appesantito enormemente il progetto senza alcun beneficio pratico su una singola macchina;
2. **Focus sul database NoSQL:** il nostro livello Gold non risiede su file Parquet in un object storage S3, ma dentro le **Time Series Collections di MongoDB 8.0**. Volevamo quindi studiare fino a dove si può arrivare costruendo i meccanismi di governance direttamente con gli strumenti nativi di MongoDB e con il codice Python della pipeline.

Abbiamo così realizzato un **lakehouse "artigianale"**, ricostruendo i quattro mattoni fondamentali (catalogo e stato delle transazioni, dizionario dei metadati, validazione dello schema e recupero dai crash). Nella Tabella 3.2 riassumiamo cosa offre un lakehouse industriale, come lo abbiamo emulato noi e quale limite paghiamo per questa scelta.

Funzionalità Lakehouse	Nostra implementazione (MongoDB + Python)	Limite rispetto a Delta Lake / Iceberg
Transaction Log e isolamento lettori	Collezione `registry` con stato `loading` → `committed` / `failed` e filtro `{"status": "committed"}` su tutti i lettori.	Non è una transazione ACID nativa del motore DB: se il processo muore a metà, i punti restano in `vital_signals` finché non gira `recover_incomplete_loads`.
Schema Enforcement e Data Contract	`$jsonSchema` nativo su `registry` + funzione Python `validate_records(records)` prima di `insert_many` su `vital_signals`.	MongoDB 8.0 vieta `$jsonSchema` sulle collezioni Time Series; se qualcuno inserisse dati da `mongosh` aggirando Python, il DB non bloccherebbe valori fuori range.
Catalogo metadati e Single Source of Truth	Dizionario `TRACK_SPECS` in `src/config.py` sincronizzato in `track_metadata` da `seed_track_metadata(db)`.	Sincronizzazione unidirezionale all'avvio di `load_mongo.py`, non integrata in un metastore esterno (es. Hive Metastore / Unity Catalog).
Data Lineage e storicizzazione	`run_id` e `pipeline_version` condivisi nei 4 JSON e nel `registry`; backup dei report in `quality_history/`.	Manca il *time travel* sui dati (sovrascrivendo un caso non si può interrogare la versione precedente) e `run_id` non è copiato dentro ogni singolo documento di `vital_signals`.

Tabella 3.2: Confronto tra le garanzie di un data lakehouse industriale e la nostra emulazione applicativa su MongoDB 8.0.

3.3.1 Catalogo, macchina a stati e provenienza (collezione registry)

La collezione `registry` svolge il ruolo di catalogo centrale dei casi caricati nel layer Gold. Ogni documento generato da `build_registry_doc` in `src/gold/load_mongo.py` registra:

- `case_id` e `schema_version` (`"1.0"`);
- `status`: lo stato del caricamento, che può valere `"loading"`, `"committed"` oppure `"failed"`;
- `record_count`: il numero di secondi effettivamente inseriti in `vital_signals`;
- `run_id` e `pipeline_version` (`"1.1.0"`), sia a livello radice sia dentro `provenance`;
- `provenance`: un sotto-documento che riporta lo step (`"silver_to_gold"`), il `timestamp` UTC e il dizionario `cleaning_params` con i parametri usati nella pulizia (`max_gap_seconds: 5.0`, `nibp_staleness_limit_s: 300.0`, `sampling_interval_s: 1.0`).

A livello di database, lo script di inizializzazione `mongo/registry_schema.js` applica alla collezione `registry` una regola di **JSON Schema Validation** con `validationLevel: "strict"` e `validationAction: "error"` (Listato 3.1), che rifiuta qualsiasi documento privo dei campi strutturali obbligatori `case_id`, `schema_version` e `provenance`.

```
// Schema definito in mongo/registry_schema.js
db.runCommand({
    collMod: "registry",
    validator: {
        $jsonSchema: {
            bsonType: "object",
```

continua dalla pagina precedente...

```
            required: ["case_id", "schema_version", "provenance"],
            properties: {
                case_id: {
                    bsonType: ["int", "long"],
                    description: "ID del caso VitalDB, richiesto e di tipo intero (32 o 64 bit)"
                },
                schema_version: {
                    bsonType: "string",
                    description: "Versione dello schema, richiesto e di tipo stringa"
                },
                record_count: {
                    bsonType: ["int", "long"],
                    description: "Numero di record caricati per questo caso"
                },
                provenance: {
                    bsonType: "object",
                    required: ["step", "timestamp"],
                    properties: {
                        step: { bsonType: "string" },
                        timestamp: { bsonType: "date" }
                    }
                }
            }
        }
    },
    validationLevel: "strict",
    validationAction: "error"
});
```

Listato 3.1: Validazione JSON Schema applicata alla collezione registry in mongo/registry_schema.js.

3.3.2 Catalogo delle tracce e sincronizzazione automatica (track_metadata)

Accanto a `registry`, abbiamo creato la collezione `track_metadata`, che documenta per ciascuna delle cinque tracce del monitor `Solar8000` l'unità di misura (`bpm`, `%`, `mmHg`), la descrizione e l'intervallo fisiologico atteso (`expected_range`).
All'avvio del container MongoDB, `mongo/registry_schema.js` popola `track_metadata` con i valori iniziali. Tuttavia, poiché Docker esegue gli script in `/docker-entrypoint-initdb.d/` solo la prima volta che crea il volume vuoto, se in seguito cambiassimo un limite fisiologico in Python il database resterebbe con il valore vecchio. Per evitare questo disallineamento, all'inizio di `process_silver_to_gold` chiamiamo sempre la funzione `seed_track_metadata(db)`, che rilegge `config.TRACK_SPECS` ed esegue un `update_one(..., upsert=True)` per ogni traccia, mantenendo `track_metadata` allineata al codice.

3.3.3 Limiti transazionali delle Time Series, protocollo a due fasi e recovery

Lavorando con MongoDB 8.0 ci siamo scontrati con due limiti tecnici delle **Time Series Collections**:

1. **Impossibilità di usare transazioni ACID multi-documento:** MongoDB non permette di

scrivere in una collezione Time Series dentro una sessione transazionale; se ci si prova, il server restituisce l'errore nativo 263 (`OperationNotSupportedInTransaction`). Non potevamo quindi inserire i punti in `vital_signals` e aggiornare `registry` dentro un'unica transazione atomica;

2. **Impossibilità di applicare `$jsonSchema` a `vital_signals`:** sulle collezioni Time Series MongoDB non accetta validatori di schema lato server.

Per aggirare questi due ostacoli senza cambiare database, in `src/gold/load_mongo.py` abbiamo implementato un protocollo applicativo a due fasi (*Two-Phase Load*) combinato con una validazione in Python:

- **Validazione preventiva (`validate_records(records)`):** prima di toccare il database, la funzione controlla in memoria tutti i documenti del paziente, verificando che `timestamp` sia un `datetime` valido, che `metadata.case_id` sia un intero nativo Python, che `metadata.sensor_name` sia una stringa e che ogni valore in `metrics` sia un numero finito non-`NaN`. Se anche un solo record è corrotto, solleva un `ValueError` prima di iniziare la scrittura;
- **Fase 1 – Stato `loading` e pulizia preventiva:** per ogni caso da caricare, `load_case_to_mongo` scrive prima nel `registry` un documento con `status: "loading"` e `record_count: 0`, e subito dopo esegue `db['vital_signals'].delete_many({"metadata.case_id": int(case_id)})` per rimuovere eventuali residui di caricamenti precedenti;
- **Fase 2 – Inserimento e promozione a `committed` (o `failed`):** la funzione esegue `db['vital_signals'].insert_many(records)` per l'intero caso. Se l'inserimento va a buon fine, aggiorna il documento in `registry` impostando `status: "committed"`, il `record_count` finale e l'oggetto `provenance` completo. Se invece `insert_many` solleva un'eccezione `PyMongoError`, il blocco `except` cancella subito i punti parziali appena scritti in `vital_signals` e imposta `status: "failed"` (con il messaggio `last_error`) nel `registry`;
- **Bonifica automatica all'avvio (`recover_incomplete_loads`):** se il container viene fermato bruscamente (es. `SIGKILL` o caduta di corrente) mentre un caso è in stato `loading`, il blocco `except` non fa in tempo a girare. Per questo motivo, ad ogni avvio di `load_mongo.py`, la funzione `recover_incomplete_loads(db, active_case_ids)` cerca nel `registry` tutti i casi con `status != "committed"` e cancella i loro punti da `vital_signals`. Inoltre, la stessa funzione controlla se nel database sono rimasti casi che non fanno più parte del perimetro Silver corrente (`active_case_ids`) e li rimuove da `vital_signals`, `registry` e `anomalies_detected`;
- **Filtro `status: "committed"` su tutti i lettori:** tutti gli script che leggono il layer Gold — `list_loaded_cases` in `src/analysis/queries.py`, `detector.py`, `evaluate_models.py` e `benchmark_etl.py` — interrogano `registry` filtrando rigorosamente `{"status": "committed"}` (esteso con operatore `$or` a `{"status": {"$exists": false}}` per preservare retrocompatibilità con eventuali record caricati in prove preliminari), così da escludere a priori qualsiasi caso rimasto nello stato `loading` o `failed`.

3.4 Pipeline di aggregazione e ottimizzazione delle query

Per mantenere reattiva la dashboard web anche su interventi chirurgici che durano molte ore, abbiamo spostato il lavoro di filtraggio e riduzione dei punti direttamente dentro il motore di aggregazione di MongoDB (`src/analysis/queries.py`).

3.4.1 Downsampling lato database con $dateTrunc

Un intervento di cinque o sei ore contiene oltre 20.000 punti a 1 Hz. Inviare sempre tutti i punti grezzi via HTTP al browser appesantirebbe il rendering dei grafici Chart.js quando l'utente vuole solo vedere l'andamento generale dell'operazione.

La funzione `downsample_case(db, case_id, window_seconds)` risolve il problema eseguendo sul database una pipeline a tre stadi (Listato 3.2):

1. `$match` seleziona i soli bucket del paziente richiesto tramite `metadata.case_id`;
2. `$group` usa l'operatore temporale `$dateTrunc` per raggruppare i timestamp in finestre fisse di ampiezza `window_seconds` (30, 60 o 300 secondi) e calcola con `$avg` la media di ciascuno dei cinque parametri vitali nella finestra;
3. `$project` e `$sort` rinominano la chiave di gruppo in `window_start` e restituiscono le finestre ordinate nel tempo.

```
// Pipeline eseguita da downsample_case() in src/analysis/queries.py
pipeline = [
    {"$match": {"metadata.case_id": case_id}},
    {
        "$group": {
            "_id": {
                "$dateTrunc": {
                    "date": "$timestamp",
                    "unit": "second",
                    "binSize": window_seconds
                }
            },
            "Solar8000_HR":          {"$avg": "$metrics.Solar8000_HR"},
            "Solar8000_PLETH_SPO2": {"$avg": "$metrics.Solar8000_PLETH_SPO2
    "},
            "Solar8000_NIBP_SBP":    {"$avg": "$metrics.Solar8000_NIBP_SBP"
    },
            "Solar8000_NIBP_DBP":    {"$avg": "$metrics.Solar8000_NIBP_DBP"
    },
            "Solar8000_NIBP_MBP":    {"$avg": "$metrics.Solar8000_NIBP_MBP"
    },
        }
    },
    {"$project": {"_id": 0, "window_start": "$_id", ...}},
    {"$sort": {"window_start": 1}}
]
```

Listato 3.2: Pipeline di aggregazione MongoDB per il downsampling temporale in src/analysis/queries.py.

3.4.2 Eliminazione del problema N+1 sull'elenco dei pazienti (e relativo limite)

Quando la dashboard si apre, chiama l'endpoint `GET /cases` per popolare la barra laterale con i 500 pazienti, mostrando per ciascuno l'ID, il reparto chirurgico, l'età e il sesso. Poiché nel `registry` sono salvati solo `case_id` e `record_count`, un'implementazione ingenua farebbe prima una query su `registry` e poi 500 query `find_one` separate su `vital_signals` per leggere i metadati di ogni paziente ($N + 1 = 501$ query totali).

Per evitare 501 chiamate di rete, in `api/main.py` abbiamo ridotto l'operazione a **due sole query**:

1. una chiamata a `list_loaded_cases(db)` che legge in blocco i 500 casi `committed` da `registry`;
2. una singola aggregazione su `vital_signals` con `$group` su `$metadata.case_id` e `$first` sui campi `department`, `age` e `sex`, unendo poi i due risultati in memoria in Python.

Limite di questa soluzione: anche se evita le 500 query separate, lo stadio `$group` su

`vital_signals` costringe comunque MongoDB a scorrere i bucket della collezione Time Series ad ogni apertura della pagina. Con il senno di poi, una soluzione ancora più leggera sarebbe stata salvare `department`, `age` e `sex` direttamente dentro il documento della collezione `registry` durante il caricamento Gold, così da servire `GET /cases` leggendo unicamente i 500 documenti del catalogo.

3.5 Metriche di data quality per sensore e analisi critica

Oltre ai flag salvati dentro ciascun documento BSON (`quality_flags`), la pipeline produce alla fine dello step Silver il report `data/quality_report.json`.
Durante la revisione dei risultati ci siamo accorti che nella prima versione del report i conteggi non avevano tutti lo stesso denominatore: i valori osservati venivano contati sul file Bronze grezzo, mentre i valori mancanti venivano contati dopo aver scartato le righe vuote, e per le tre pressioni NIBP il numero di imputati veniva stimato indirettamente da `nibp_age_s > 0` invece che contato su ogni singolo canale. Abbiamo quindi riscritto la sezione `sensor_metrics` di `clean.py` (righe 288–312) calcolando tutti gli indicatori sullo stesso perimetro di righe pulite `rows_cleaned` (`rows_after`). Per ognuna delle cinque tracce vale ora l'identità esatta:

$$\texttt{observed} + \texttt{imputed} + \texttt{missing} \equiv \texttt{rows_cleaned}$$

dove `imputed` è la somma del flag booleano dedicato (`HR_imputed`, `SPO2_imputed`, `SBP_imputed`, `DBP_imputed`, `MBP_imputed`), `observed` conta le celle non nulle e non imputate, e `missing` conta i `NaN` residui. A questi conteggi sono affiancate tre percentuali:

- $\texttt{observed_pct} = \frac{\texttt{observed}}{\texttt{rows_cleaned}} \times 100$: quota di misure realmente lette dal sensore sulle righe attive;
- $\texttt{coverage_pct} = \frac{\texttt{observed}+\texttt{imputed}}{\texttt{rows_cleaned}} \times 100$: copertura effettiva disponibile in Silver/Gold dopo l'interpolazione e il LOCF;
- $\texttt{temporal_coverage_bronze_pct} = \frac{\texttt{observed}+\texttt{imputed}}{\texttt{rows_original}} \times 100$: copertura rapportata alla durata grezza iniziale del file Bronze (`rows_before`).

Chiudiamo il capitolo riassumendo tre limiti strutturali della nostra gestione della qualità e del layer Gold:

1. **Qualità controllata per riga ma non per intero caso:** grazie ai flag in `quality_flags` evitiamo di scartare un intero intervento se un sensore si stacca per qualche minuto, ma l'assenza di una soglia minima bloccante (*case-level quality gate*) fa sì che anche casi con copertura molto bassa vengano comunque caricati in Gold;
2. **Lineage fermata al catalogo `registry`:** l'identificativo `run_id` è salvato nel `registry` e nei file JSON, ma non dentro ogni singolo documento di `vital_signals` (scelta fatta per non gonfiare i documenti Time Series). Tutte le letture passano quindi da `registry` per sapere quali casi sono validi;
3. **Asse temporale convenzionale (`2020-01-01 + Time`):** poiché VitalDB fornisce solo i secondi relativi dall'inizio della registrazione per ragioni di anonimizzazione, tutte le serie Gold partono dalla data fittizia del 1° gennaio 2020. Questo non influisce né sulla compressione Delta-of-Delta né sui modelli, ma in un sistema ospedaliero reale verrebbe sostituito dall'orario effettivo del monitor.

CAPITOLO 4

Modelli di anomaly detection

4.1 Perché un approccio ibrido: regole cliniche + machine learning non supervisionato

Nel monitoraggio intraoperatorio continuo, un paziente può destabilizzarsi in modi molto diversi: talvolta con un crollo acuto e netto di un singolo parametro (ad esempio una caduta brusca della pressione arteriosa o della saturazione), altre volte con variazioni più lente e distribuite su più canali contemporaneamente.

Quando abbiamo progettato il modulo di rilevamento delle anomalie (`src/detection/detector.py`), ci siamo trovati davanti a un primo problema pratico: **nel dataset pubblico VitalDB non esistono etichette cliniche secondo-per-secondo** che indichino quando il paziente sta attraversando un evento avverso certificato da un anestesista. Questa assenza di annotazioni puntuali escludeva a priori l'addestramento di classificatori supervisionati classici (come Random Forest o XGBoost addestrati su etichette vere).

D'altra parte, affidarsi solo a regole cliniche a soglia fissa oppure solo ad algoritmi non supervisionati presentava limiti opposti:

- le **regole cliniche deterministiche** hanno un significato medico immediato e trasparente (l'anestesista sa subito perché è scattato l'allarme), ma vedono solo una o due variabili alla volta e si attivano solo quando la soglia critica è già stata superata;
- i **modelli di machine learning non supervisionato** possono analizzare contemporaneamente tutte le variabili e le loro derivate temporali senza bisogno di etichette, ma non conoscono il significato fisiologico dei numeri: producono un punteggio di atipicità statistica che, da solo, è più difficile da interpretare al letto del paziente.

Per questo motivo abbiamo scelto un **approccio ibrido a quattro metodi complementari**:

1. due **regole cliniche dominio-specifiche**: lo **Shock Index** (SI $> 0,9$) e l'**ipotensione severa concomitante a ipossiemia** (MBP < 65 mmHg $\wedge$ $SpO_2 < 90\%$);
2. due **modelli di machine learning non supervisionato** sullo spazio multivariato a 8 feature: l'algoritmo ad alberi **Isolation Forest** (che isola i punti anomali istantanei nello spazio a 8 dimensioni) e una rete neurale ricorrente **LSTM Autoencoder** in PyTorch (che valuta la ricostruzione di finestre scorrevoli di 30 secondi).

Affiancare un modello ad alberi a un modello ricorrente ci ha permesso di confrontare un approccio veloce basato sulla densità puntuale (Isolation Forest) con uno capace di considerare l'andamento sequenziale negli ultimi 30 secondi (LSTM Autoencoder).

4.2 Caricamento dal layer Gold e regole cliniche con filtro di freschezza

Prima di calcolare qualsiasi regola o feature, la funzione `load_from_gold(db, case_ids)` in `src/detection/detector.py` estrae le serie dal layer **Gold** applicando tre accorgimenti importanti sia per la stabilità della memoria sia per la qualità dei dati:

1. **Filtro sui soli casi `committed` e lettura a batch da 25 casi:** se non viene passata una lista esplicita di casi, la funzione interroga il catalogo `registry` prelevando esclusivamente gli identificativi con stato `{"status": "committed"}`. Inoltre, quando il numero di pazienti da caricare supera i 25 (come nell'esecuzione completa sui 500 casi), l'estrazione da `vital_signals` viene suddivisa in blocchi da `batch_size = 25` casi tramite la funzione ausilia-

ria `_fetch_gold_batch`, ordinando ogni blocco per `("metadata.case_id", 1)` e `("timestamp", 1)`. Abbiamo introdotto questo caricamento a blocchi perché una singola query su 5,4 milioni di documenti BSON causava picchi eccessivi di memoria nel cursore PyMongo;
2. **Mascheramento attivo degli outlier fisiologici:** come visto nel Capitolo 2, il layer **Silver** conserva i valori grezzi fuori scala per finalità di audit e si limita a marcarli con i flag booleani `quality_flags.*_outlier`. Se passassimo quei valori direttamente alle regole o ai modelli, un singolo artefatto da bisturi elettrico (HR $= 320$ bpm) o uno sgonfiaggio del bracciale pressorio farebbe scattare falsi allarmi. Per evitarlo, `load_from_gold()` controlla ciascuno dei 5 flag `*_outlier` e sostituisce in memoria i valori corrispondenti con `np.nan` prima di avviare l'analisi;
3. **Vincolo di freschezza clinica della pressione (`nibp_age_s` ≤ 120 s):** nel layer **Silver** le misure intermittenti del bracciale pressorio vengono propagate in avanti tramite **forward-fill** fino a 300 s (5 minuti) per non lasciare buchi continui nella matrice usata dai modelli multivariati. Tuttavia, dal punto di vista medico, fare scattare un allarme acuto di Shock Index o di ipotensione severa basandosi su una pressione misurata quattro o cinque minuti prima non sarebbe corretto: un singolo episodio ipotensivo già risolto continuerebbe a generare un allarme fittizio per 300 secondi consecutivi. Per evitare questo effetto, entrambe le regole cliniche leggono il contatore `quality_flags.nibp_age_s` e applicano la soglia configurata in `config.NIBP_MAX_CLINICAL_AGE_S = 120.0` secondi: una misura NIBP viene usata dalle regole cliniche solo se è stata acquisita negli ultimi 2 minuti.

4.2.1 Shock Index clinico

Lo **Shock Index** (SI) è un indicatore emodinamico adimensionale molto usato in anestesia e rianimazione per individuare precocemente stati di shock ipovolemico o emorragico. È definito come il rapporto tra la frequenza cardiaca (HR) e la pressione arteriosa sistolica (SBP):

$$\text{SI} = \frac{\text{Solar8000_HR}}{\text{Solar8000_NIBP_SBP}}$$

In un adulto stabile il valore normale oscilla tra $0,5$ e $0,7$. Quando il volume circolante cala (ad esempio per un sanguinamento intraoperatorio) o si verifica una forte vasodilatazione da anestetici, l'organismo compensa aumentando la frequenza cardiaca mentre la pressione sistolica scende, facendo salire il rapporto. La funzione `compute_shock_index(df, threshold=0.9, max_nibp_age_s=None)` marca come anomalo ogni secondo in cui:

$$\text{SI} > 0,9 \quad \wedge \quad \texttt{nibp_age_s} \leq 120 \text{ s}$$

Inoltre, per evitare divisioni per zero o valori negativi privi di senso, i campioni con SBP ≤ 0 vengono sostituiti con `np.nan` prima di calcolare il rapporto.

4.2.2 Ipotensione severa concomitante a ipossiemia

La seconda regola clinica, implementata in `compute_severe_hypotension(df, mbp_threshold=65.0, spo2_threshold=90.0, max_nibp_age_s=None)`, intercetta gli episodi in cui una perfusione arteriosa insufficiente si associa contemporaneamente a una caduta della saturazione periferica d'ossigeno:

$$\text{Solar8000_NIBP_MBP} < 65 \text{ mmHg} \quad \wedge \quad \text{Solar8000_PLETH_SPO2} < 90\% \quad \wedge \quad \texttt{nibp_age_s} \leq 120 \text{ s}$$

In anestesia, mantenere la pressione arteriosa media (MBP) sopra i 65 mmHg è considerato il requisito minimo per garantire la perfusione degli organi vitali (cervello, cuore e reni). Quando sotto questa soglia si verifica anche una desaturazione sotto il 90%, il paziente entra rapidamente in sofferenza ipossica-ischemica.

4.3 Ingegnerizzazione delle feature temporali sulla frequenza cardiaca

Se passassimo ai modelli di machine learning soltanto i 5 segnali vitali istantanei, un algoritmo come Isolation Forest (che non ha memoria storica) vedrebbe ogni secondo come scollegato dal precedente e non distinguerebbe una frequenza cardiaca stabile a 95 bpm da una che è appena balzata da 60 a 95 bpm in due secondi.
Per dare anche ai modelli una misura della dinamica a brevissimo termine senza esplodere il numero di colonne, nella funzione `run_detection_pipeline` abbiamo calcolato tre feature derivate a partire dal segnale più continuo e regolare del dataset, la frequenza cardiaca (`Solar8000_HR`, campionata a 1 Hz). Indicando con x_t il valore di HR al secondo t e usando una finestra retrospettiva di $W = 5$ campioni (`rolling(5, min_periods=1)`), calcoliamo:

- la **media mobile a 5 secondi** (`HR_rolling_mean`), che smorza le oscillazioni istantanee:

$$\mu_t = \frac{1}{W} \sum_{k=0}^{W-1} x_{t-k}$$

- la **deviazione standard mobile a 5 secondi** (`HR_rolling_std`, con `.fillna(0.0)` sul primo campione), che misura l'instabilità locale del battito:

$$\sigma_t = \sqrt{\frac{1}{W-1} \sum_{k=0}^{W-1} (x_{t-k} - \mu_t)^2}$$

- la **differenza prima** (`HR_delta`, calcolata con `.diff().fillna(0.0)`), che misura l'accelerazione o la decelerazione istantanea del battito da un secondo al successivo:

$$\Delta x_t = x_t - x_{t-1}$$

Tutte e tre le trasformazioni sono calcolate raggruppando esplicitamente per paziente tramite `df.groupby("case_id", sort=False)["Solar8000_HR"]`. Questo evita che i primi 4 secondi di un paziente vengano calcolati facendo media o differenza con gli ultimi secondi del paziente precedente.
Il vettore finale passato ai modelli di machine learning è quindi composto da **8 feature numeriche**: i 5 canali biometrici (`Solar8000_HR`, `Solar8000_PLETH_SPO2`, `Solar8000_NIBP_SBP`, `Solar8000_NIBP_DBP`, `Solar8000_NIBP_MBP`) più le 3 feature dinamiche (`HR_rolling_mean`, `HR_rolling_std`, `HR_delta`). Abbiamo invece escluso dal vettore delle feature `nibp_age_s`, perché rappresenta l'età tecnica della misura del bracciale e non una grandezza fisiologica del paziente.

4.4 Modelli di machine learning non supervisionato

4.4.1 Imputazione locale per paziente e correzione del bug di data leakage nel `.bfill()`

Poiché né `IsolationForest` di scikit-learn né i tensori di PyTorch accettano valori `NaN` in ingresso, prima di eseguire i modelli dobbiamo riempire gli eventuali buchi rimasti dopo la pulizia Silver (ad esempio i tratti in cui il bracciale NIBP è rimasto spento per più di 300 secondi o i punti appena mascherati a `NaN` perché outlier).
Durante la revisione del codice ci siamo accorti di un bug insidioso che avevamo introdotto nella prima versione di `detector.py` e `evaluate_models.py`. Avevamo scritto:

```
df.groupby("case_id")[feat_cols].ffill().bfill()
```

Il problema è nel comportamento di Pandas: il metodo `.ffill()` invocato su un oggetto `GroupBy` viene eseguito correttamente dentro ciascun paziente, ma restituisce un normale `DataFrame` non più raggruppato. Di conseguenza, il successivo `.bfill()` agiva sull'intero DataFrame concatenato: se un paziente iniziava l'intervento senza ancora una lettura NIBP valida nei primi minuti, quei `NaN` iniziali potevano essere riempiti all'indietro con i primi valori del paziente successivo se l'intero canale era vuoto.
Abbiamo corretto il bug sia in `detect_isolation_forest`, sia in `LSTMAutoencoderDetector._make_window_views_per_case`, sia in `evaluate_models.py`, riapplicando esplicitamente il raggruppamento per `case_id` anche sul **backward-fill**:

```
df.groupby("case_id", sort=False)[feat_cols].ffill().groupby(df["case_id"], sort=False).bfill().fillna(0.0)
```

In questo modo sia la propagazione in avanti sia quella all'indietro restano confinate dentro il singolo caso clinico, e il fallback a `0.0` interviene solo nel caso limite in cui un canale sia assente per tutta la durata di quell'intervento. Per essere sicuri di non reintrodurre l'errore, abbiamo aggiunto il test unitario dedicato `test_imputation_no_bfill_leakage_across_cases` in `tests/test_detection.py`.

4.4.2 Isolation Forest e significato della soglia `contamination=0.05`

Il primo modello non supervisionato, implementato nella funzione `detect_isolation_forest(df, contamination=0.05)`, utilizza l'algoritmo **Isolation Forest** di scikit-learn (`random_state=42`).
L'idea dell'algoritmo è semplice e molto adatta a dataset grandi: invece di stimare la densità dei punti normali (operazione computazionalmente costosa su 5,4 milioni di righe), costruisce una foresta di alberi binari (*iTree*) che partizionano lo spazio a 8 dimensioni scegliendo casualmente una variabile e un punto di taglio. I punti normali, trovandosi raggruppati in zone dense dello spazio, richiedono molti tagli successivi per essere isolati; i punti anomali, che si trovano in zone periferiche o presentano combinazioni insolite tra le 8 feature (ad esempio HR molto instabile associata a pressione bassa), vengono isolati vicino alla radice dell'albero in pochissimi passaggi. La lunghezza media del percorso dalla radice alla foglia determina lo score di anomalia del punto.
È importante chiarire con onestà cosa significa aver impostato `contamination=0.05` (e, in modo analogo, il 95° percentile nell'Autoencoder). Con questo parametro **il modello non "scopre" da solo che il 5% dei punti è anomalo**: per costruzione, l'algoritmo ordina tutti i punti dal più normale al più estremo e poi applica una soglia che etichetta come anomali esattamente il 5% dei campioni con il punteggio peggiore. In pratica usiamo entrambi i modelli non supervisionati come **meccanismi di ranking di severità** a cui chiediamo di restituire la coda del 5% più atipica. Quando viene eseguito sull'intera coorte di 500 pazienti, il taglio del 5% è globale e quindi i pazienti più instabili accumulano più segnalazioni rispetto a quelli stabili; quando invece viene eseguito su un singolo paziente, selezionerà comunque circa il 5% dei punti di quel caso.

4.4.3 LSTM Autoencoder in PyTorch e finestre scorrevoli zero-copy

Mentre Isolation Forest valuta ogni secondo in modo puntuale (aiutato solo dalle 3 feature mobili a 5 secondi su HR), il secondo modello è pensato per catturare la dinamica temporale su un orizzonte più ampio: un **LSTM Autoencoder** sviluppato in PyTorch, che analizza finestre scorrevoli di **30 secondi consecutivi** su tutti gli 8 canali.
Come illustrato nella Figura 4.1 e nel Listato 4.1, la classe `_LSTMAutoencoder` (costruita dalla factory `_make_lstm_autoencoder_class()` in `src/detection/detector.py`) ha una struttura **encoder-decoder**:

- **Input:** riceve un tensore di forma $(B, 30, 8)$, dove B è la dimensione del batch, `window_size=30` sono i 30 secondi consecutivi e 8 sono le feature standardizzate con `StandardScaler` (media zero e varianza unitaria);

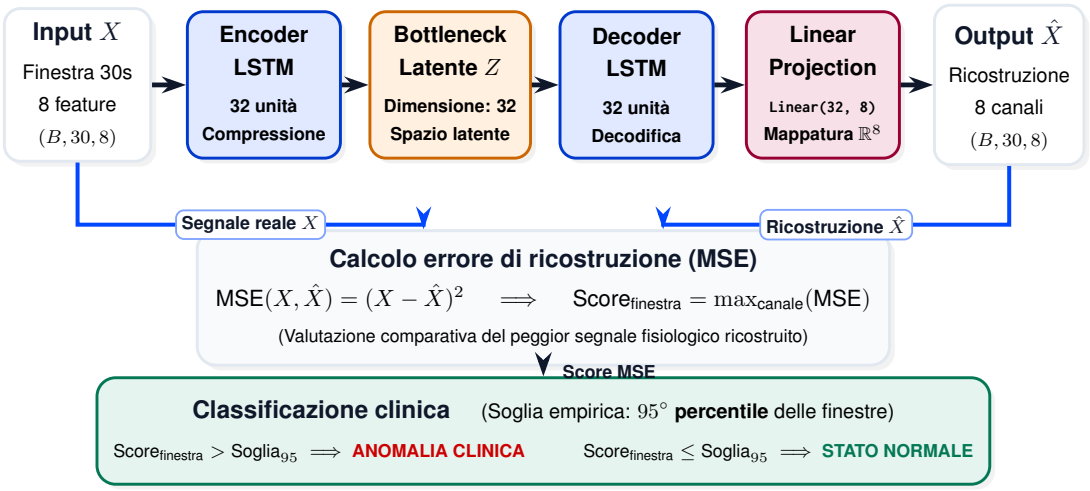


Figura 4.1: Architettura della rete neurale LSTM Autoencoder con proiettore lineare e calcolo dell'errore massimo per canale.

- **Encoder LSTM:** uno strato `nn.LSTM(n_features=8, hidden_size=32, batch_first=True)` legge i 30 passi temporali e comprime la sequenza nello stato nascosto finale `hidden` di dimensione 32 (*bottleneck*);
- **Repeat vector e Decoder LSTM:** il vettore latente di dimensione 32 viene ripetuto per tutti i 30 passi temporali (`hidden.permute(1, 0, 2).repeat(1, x.size(1), 1)`) e passato al decoder `nn.LSTM(32, 32, batch_first=True)`;
- **Proiettore lineare (`nn.Linear(32, 8)`):** all'uscita del decoder abbiamo aggiunto uno strato lineare `self.out = nn.Linear(hidden_size, n_features)`. Questa scelta è stata fondamentale: l'uscita nativa di una cella LSTM passa attraverso una tangente iperbolica ($\tanh$), che restituisce solo valori compresi tra -1 e $+1$. Poiché le nostre feature sono standardizzate in Z-score e possono tranquillamente assumere valori come $+2,5$ o $-3,0$, senza il layer `nn.Linear` finale la rete sarebbe stata fisicamente incapace di ricostruire i valori oltre ± 1.

```python
def _make_lstm_autoencoder_class():
    """Costruisce la classe LSTMAutoencoder come nn.Module solo se PyTorch e' disponibile."""
    if not _TORCH_AVAILABLE:
        return None

    class _LSTMAutoencoder(nn.Module):
        def __init__(self, n_features, hidden_size=32, n_layers=1):
            super().__init__()
            # Encoder LSTM: comprime la sequenza in un vettore latente
            self.encoder = nn.LSTM(n_features, hidden_size, n_layers, batch_first=True)
            # Decoder LSTM: decodifica il vettore latente nello spazio nascosto
            self.decoder = nn.LSTM(hidden_size, hidden_size, n_layers, batch_first=True)
            # Layer di proiezione lineare: mappa lo stato nascosto nello spazio continuo R^n
            self.out = nn.Linear(hidden_size, n_features)

```

continua dalla pagina precedente...

```
        def forward(self, x):
            # x ha forma (batch, T, n_features)
            _, (hidden, _) = self.encoder(x)
            # Ripete il vettore latente per ogni timestep della sequenza
            context = hidden.permute(1, 0, 2).repeat(1, x.size(1), 1)
            output, _ = self.decoder(context)
            return self.out(output)  # dimensione: (batch, T, n_features)

    return _LSTMAutoencoder
```

Listato 4.1: Definizione reale della classe PyTorch `_LSTMAutoencoder` in `src/detection/detector.py`.

Gestione della memoria sui 500 casi: viste zero-copy e sottocampionamento di training

Quando siamo passati dai primi test su 50 casi all'intera coorte di **500 casi** (**5.402.386 righe**), ci siamo scontrati con un problema concreto di memoria RAM nella classe `LSTMAutoencoderDetector`. Creare una finestra scorrevole di lunghezza 30 e passo 1 su $5,4$ milioni di punti significa generare circa $5,39$ milioni di finestre di forma $(30, 8)$. Se avessimo concatenato tutte queste finestre in un unico array NumPy tridimensionale tramite `np.concatenate`, avremmo dovuto allocare di colpo oltre **5,17 GB** di RAM in `float32` (o oltre **10,3 GB** in `float64`), mandando in crash il container Docker per *Out-Of-Memory*.
Per far girare il modello su CPU con un consumo di memoria ridotto, in `LSTMAutoencoderDetector` abbiamo adottato tre soluzioni pratiche:

1. **Viste zero-copy per paziente (`_make_window_views_per_case`):** dopo aver normalizzato le 8 feature con `StandardScaler` e averle convertite subito in `np.float32`, per ciascun paziente costruiamo le finestre con `np.lib.stride_tricks.sliding_window_view(vals, window_shape=(30, 8))`. Questa funzione di NumPy non copia i dati in memoria, ma crea una vista virtuale modificando solo gli *strides* (i passi di lettura dei puntatori) sull'array 2D già esistente, evitando sia la duplicazione di 5 GB sia la creazione di finestre a cavallo tra due pazienti diversi;
2. **Sottocampionamento uniforme per il training (`max_train = 15000`):** addestrare una rete LSTM su CPU per 10 epoche su 5,39 milioni di finestre richiederebbe ore. Nel metodo `fit_predict`, se il totale delle finestre supera `max_train = 15000`, estraiamo dalle viste zero-copy un campione uniforme di 15.000 finestre distribuite lungo tutta la coorte (`np.linspace(0, total_windows - 1, max_train, dtype=int)`), che occupano appena **14,4 MB** di RAM, e riduciamo il numero di epoche a `n_epochs = min(self.epochs, 5)` (cioè 5 epoche, con batch size dinamico `batch_sz = min(512, max(32, len(train_tensor)))` e ottimizzatore Adam con `lr=1e-3`). Se invece le finestre sono ≤ 15.000 (ad esempio sul singolo paziente), il training usa tutte le finestre per `self.epochs = 10` epoche;
3. **Inferenza a blocchi (`eval_batch_size = 4096`):** una volta addestrata la rete, la valutazione dell'errore di ricostruzione viene eseguita su **tutte** le finestre di tutti i pazienti, ma scorrendo le viste zero-copy un blocco da `eval_batch_size = 4096` finestre alla volta dentro `torch.no_grad()`, senza mai materializzare l'intero tensore 3D in memoria.

Scelta dell'errore massimo tra i canali ($\max_c$ MSE$_c$)

Un'altra scelta tecnica importante riguarda il modo in cui calcoliamo l'errore di ricostruzione di una finestra. Nella prima versione calcolavamo la media dell'errore quadratico su tutti e 8 i canali contemporaneamente. Ci siamo però accorti di un difetto evidente: poiché le 3 colonne

della pressione NIBP vengono mantenute costanti dal *forward-fill* per lunghi tratti, l'Autoencoder impara a ricostruirle quasi perfettamente con errore prossimo a zero. Di conseguenza, se un singolo segnale (ad esempio la SpO_2) subisce un crollo improvviso mentre gli altri 7 canali restano regolari, facendo la media aritmetica su 8 variabili quel picco di errore viene diviso per 8 e "annacquato", rischiando di non superare la soglia.
Per evitare questa diluizione, nel codice calcoliamo prima l'errore quadratico medio temporale sui 30 secondi per ciascuna feature (`mean_time_err = torch.mean(sq_err, dim=1)`) e poi prendiamo il **massimo errore tra gli 8 canali** (`batch_mse, _ = torch.max(mean_time_err, dim=1)`):

$$\mathcal{E}(X) = \max_{c \in \{1,\dots,8\}} \left(\frac{1}{30} \sum_{t=1}^{30} (X_{t,c} - \hat{X}_{t,c})^2 \right)$$

L'errore $\mathcal{E}(X)$ di ciascuna finestra di 30 secondi viene assegnato al suo ultimo istante temporale (`case_idx[self.window_size - 1:]`) e vengono marcati come anomali i punti che superano il $95°$ percentile (`self.percentile = 95.0`) degli errori calcolati sulle finestre. I primi 29 secondi di ciascun paziente, non avendo ancora una finestra completa di 30 campioni alle spalle, mantengono errore zero e non vengono marcati dall'Autoencoder.

Sintesi delle scelte sull'LSTM Autoencoder

L'uso del massimo errore tra i canali ($\max_c \text{MSE}_c$) evita che i canali pressori costanti in *forward-fill* nascondano la caduta acuta di un singolo parametro vitale. Allo stesso tempo, le viste `sliding_window_view` per paziente e la valutazione a blocchi da 4.096 finestre evitano di allocare oltre 5 GB di tensori 3D in RAM.

4.5 Valutazione empirica sui 500 casi e interpretazione delle metriche

Per valutare quantitativamente i due modelli non supervisionati sulla coorte di 500 pazienti abbiamo realizzato lo script `src/analysis/evaluate_models.py`, che salva i risultati in `data/ml_evaluation.json` (esposti poi dall'endpoint `GET /evaluation`).
Poiché, come detto, VitalDB non fornisce etichette di anomalia annotate da un medico, nello script abbiamo costruito una **pseudo-ground truth clinica** definendo come "positivi di riferimento" gli istanti temporali in cui scatta almeno una delle due regole cliniche deterministiche (Shock Index $> 0,9$ oppure ipotensione severa MBP $< 65 \wedge$ $SpO_2 < 90\%$, entrambe con il filtro di freschezza `nibp_age_s` ≤ 120 s). Inoltre, rispetto a una versione iniziale dello script che addestrava due volte `IsolationForest` (una chiamata a `detect_isolation_forest` per i flag binari e un secondo `fit` per gli score continui), abbiamo ottimizzato la procedura eseguendo un **singolo addestramento** dell'istanza `iso_model`, da cui ricaviamo sia le predizioni binarie (`fit_predict`) sia i punteggi continui (`-iso_model.score_samples`) usati per calcolare la curva Precision-Recall (`average_precision_score`).
Su un totale di **5.402.386 campioni** appartenenti ai **500 casi** con stato `committed`, le regole cliniche individuano **262.978 secondi positivi**, pari a una prevalenza (`base_rate_pct`) del **4,87%**. Con una classe positiva al $4,87\%$ e il restante $95,13\%$ di campioni stabili, la classica *Accuracy* sarebbe completamente fuorviante (*Accuracy Paradox*): un classificatore banale che rispondesse sempre "normale" otterrebbe un'accuratezza del **95,13%** pur non intercettando alcuna emergenza. Per questo motivo abbiamo valutato i modelli solo con metriche adatte a classi fortemente sbilanciate: **Precision**, **Recall**, **F1-score** sulla classe anomala e soprattutto **PR-AUC** (*Average Precision*), la cui baseline casuale coincide con la prevalenza della classe positiva ($0,0487$). I risultati sono riportati nella Tabella 4.1.

Modello / Algoritmo	Precision (anom.)	Recall (anom.)	F1-score (anom.)	PR-AUC (Average Precision)	Confronto con Baseline casuale
Classificatore banale (Baseline)	0,000	0,000	0,000	0,049 (Base rate 4,87%)	$1,0\times$ (Caso)
Isolation Forest	**0,124**	**0,127**	**0,126**	**0,105**	$\approx 2,2\times$ **sopra il caso**
LSTM Autoencoder	0,096	0,098	0,097	0,067	$\approx 1,4\times$ **sopra il caso**

Tabella 4.1: Risultati della valutazione dei modelli non supervisionati rispetto alla pseudo-ground truth clinica sui 500 casi Gold (5.402.386 campioni, da `data/ml_evaluation.json`).

Guardando la Tabella 4.1 si nota subito che i valori assoluti sono bassi: **Isolation Forest** ottiene una PR-AUC di **0,105** (circa $2,2$ volte la baseline casuale di $0,0487$, con F1-score $0,126$), mentre l'**LSTM Autoencoder** ottiene una PR-AUC di **0,067** (circa $1,4$ volte il caso, con F1-score $0,097$). È giusto commentare con estrema chiarezza cosa ci dicono questi numeri e perché non indicano né un bug del codice né l'inutilità dei modelli:

1. **Cosa stiamo misurando davvero (concordanza, non verità clinica assoluta):** le etichette di riferimento non sono diagnosi esterne, ma il risultato di due disuguaglianze matematiche calcolate su HR, SBP, MBP e SpO_2. Di conseguenza, queste metriche misurano solo **quanto spesso il 5% di punti giudicati più atipici dai modelli ML coincide con gli istanti in cui scattano le due regole cliniche**;
2. **Perché la sovrapposizione è bassa:** le regole cliniche guardano solo due variabili per volta in un singolo secondo e si spengono completamente non appena la misura del bracciale pressorio supera i 120 secondi di età (`nibp_age_s` > 120 s). I due modelli non supervisionati, invece, lavorano su 8 variabili che includono la deviazione standard mobile (`HR_rolling_std`), gli sbalzi istantanei del battito (`HR_delta`), la pressione diastolica (DBP) e — per l'Autoencoder — l'andamento su 30 secondi, continuando a segnalare anomalie anche nei minuti in cui il bracciale NIBP è in pausa. È quindi normale che molti punti segnalati dai modelli (ad esempio una brusca aritmia o una caduta pressoria rapida che si ferma a MBP $= 67$ mmHg, appena sopra la soglia di 65) vengano contati come "falsi positivi" rispetto alle regole statiche;
3. **Perché Isolation Forest supera l'LSTM Autoencoder su questo specifico benchmark:** poiché la pseudo-ground truth è definita istante per istante su soglie puntuali, Isolation Forest (che lavora anch'esso sul singolo secondo) è strutturalmente più allineato al modo in cui sono costruite le regole rispetto all'Autoencoder, che invece valuta l'errore medio di ricostruzione su una finestra di 30 secondi e risente anche del fatto di essere addestrato per 5 epoche su un campione di 15.000 finestre.

In sintesi, se i due modelli avessero ottenuto un F1-score vicino a $1,0$ rispetto alle regole cliniche, sarebbero stati un duplicato inutile e computazionalmente pesante di due semplici `if` matematici; il fatto che ordinino gli eventi meglio del caso ($2,2\times$ e $1,4\times$) ma segnalino anche situazioni diverse conferma che catturano pattern multivariati complementari a quelli delle soglie statiche.
La Figura 4.2 mostra la distribuzione empirica degli errori di ricostruzione dell'Autoencoder: la maggior parte delle finestre si concentra vicino allo zero, mentre la coda destra oltre la linea tratteggiata del $95°$ percentile raccoglie le sequenze di 30 secondi che la rete fatica a ricostruire.
La Figura 4.3 illustra infine la matrice di concordanza tra i quattro metodi di rilevamento.

4.6 Persistenza su MongoDB con pattern Insert-Before-Delete e limiti del modulo

Al termine dell'analisi, la funzione `save_anomalies_to_mongo(db, df_anomalies)` salva nella collezione MongoDB `anomalies_detected` esclusivamente gli istanti temporali in cui almeno uno dei quattro metodi ha restituito `True`. Ogni documento salvato contiene il `case_id`, il `timestamp`, l'array `methods` con i nomi dei metodi attivati (es. `["shock_index", "isolation_forest"]`) e il timestamp di esecuzione `detected_at`.

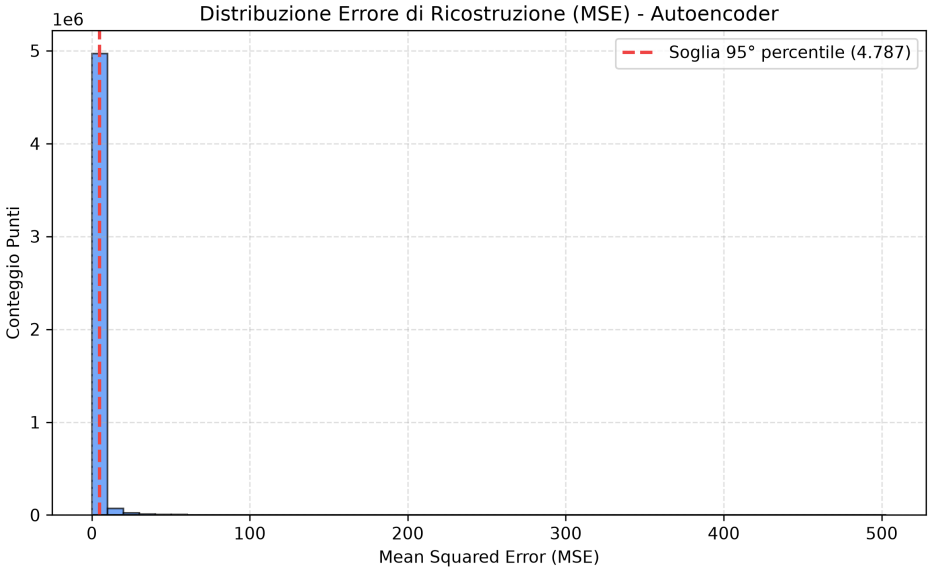


Figura 4.2: Distribuzione dell'errore di ricostruzione (MSE) dell'LSTM Autoencoder con taglio al $95°$ percentile.

Nelle prime versioni del progetto, prima di salvare le nuove anomalie cancellavamo subito tutte quelle vecchie con `delete_many`. Su un'esecuzione da 500 casi questa scelta era rischiosa: se lo script veniva interrotto a metà del calcolo o dell'inserimento, la collezione `anomalies_detected` restava vuota. Per evitare questo problema abbiamo adottato il pattern **Insert-Before-Delete**:

1. generiamo un unico timestamp UTC di batch `now = datetime.now(timezone.utc)` per tutti i nuovi documenti;
2. inseriamo prima tutti i nuovi record a blocchi di 20.000 documenti con `insert_many`;
3. **solo dopo che tutti gli inserimenti sono andati a buon fine**, cancelliamo per i casi analizzati i documenti delle esecuzioni precedenti che hanno un timestamp diverso da quello appena scritto:

   ```
   db["anomalies_detected"].delete_many({"case_id": {"$in": analyzed_cases}, "detected_at": {"$ne": now}}
   ```

   (se invece per i casi analizzati non è stata trovata alcuna anomalia, viene eseguita la pulizia con `{"case_id": {"$in": analyzed_cases}}`).

Limiti del modulo di Anomaly Detection e della valutazione ML

Anche per la parte di rilevamento delle anomalie è importante elencare con trasparenza i limiti progettuali e operativi del nostro sistema:

1. **Frazione di anomalie ML fissata a priori (5%):** avendo impostato `contamination=0.05` in Isolation Forest e `percentile=95.0` nell'LSTM Autoencoder, entrambi i modelli etichettano sempre per costruzione circa il 5% dei campioni valutati. Il numero complessivo di anomalie ML non è quindi una "scoperta" automatica della prevalenza patologica, ma il risultato di una soglia di ranking scelta da noi;
2. **Pseudo-ground truth circolare e priva di annotazioni mediche esterne:** le metriche in `ml_evaluation.json` confrontano i modelli non supervisionati con regole cliniche calcolate sulle stesse tracce vitali. Si tratta quindi di una misura di concordanza interna e non di una validazione clinica su diagnosi indipendenti;
3. **Assenza di persistenza dei pesi dei modelli (`.pkl` / `.pt`):** il progetto non salva su disco i modelli addestrati sull'intera coorte di 500 casi. Sia `IsolationForest`

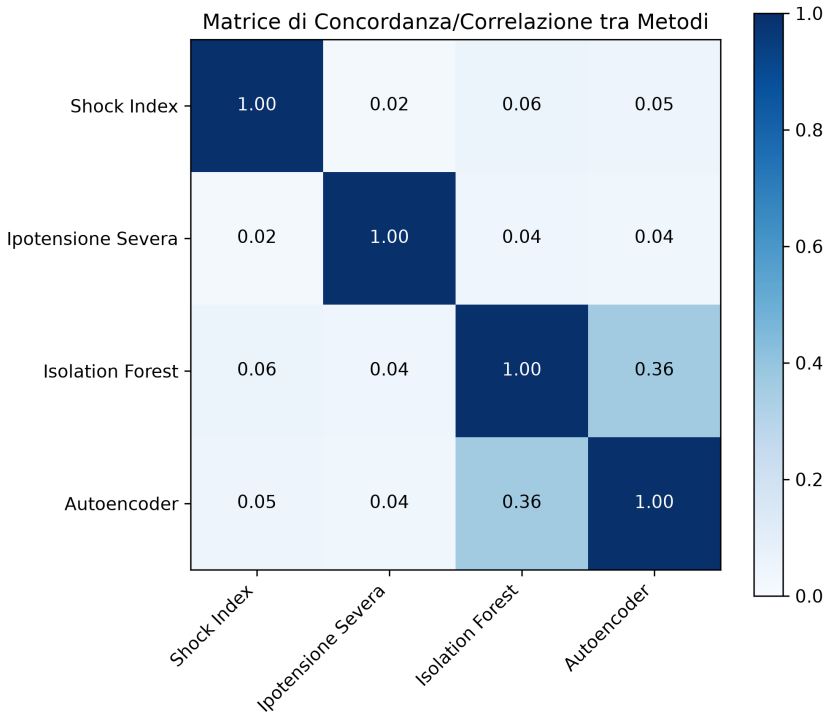


Figura 4.3: Matrice di concordanza e correlazione tra le due regole cliniche e i due modelli di machine learning.

sia `LSTMAutoencoderDetector` eseguono `fit_predict` da zero ad ogni lancio della pipeline;

4. **Differenza tra esecuzione batch (500 casi) e ricalcolo da dashboard (1 caso):** quando l'utente preme "Run Anomaly Detection" sulla dashboard per un singolo paziente, l'endpoint `POST /cases/{case_id}/detect` richiama `run_detection_pipeline(db, [case_id])`. Poiché i modelli non sono pre-addestrati e salvati, essi vengono riaddestrati da zero solo sui dati di quel singolo paziente (marcando quindi sempre il 5% dei punti di quel caso, anche se il paziente fosse perfettamente stabile) e sovrascrivono in `anomalies_detected` le anomalie che per quel caso erano state calcolate sul batch globale da 500 pazienti;
5. **Assenza di `detection_run_id` nei documenti di `anomalies_detected`:** mentre il file `ml_evaluation.json` registra il `run_id`, i singoli documenti salvati in `anomalies_detected` contengono il timestamp `detected_at` ma non un identificativo `detection_run_id` né la configurazione degli iperparametri usati.

CAPITOLO 5

Ingegnerizzazione del sistema, interfaccia grafica e conclusioni

5.1 Backend REST API con FastAPI

Per mettere in comunicazione il layer **Gold** di MongoDB, il motore di **anomaly detection** e l'interfaccia web, abbiamo sviluppato un backend REST in Python basato sul framework **FastAPI**, contenuto nel file `api/main.py` ed eseguito nel container Docker `vitaldb_api` sulla porta `8000`.
Abbiamo scelto FastAPI per tre motivi pratici:

1. permette di definire le rotte HTTP in poche righe di codice con validazione automatica dei tipi dei parametri (come `case_id: int` e `window_seconds: Optional[int]`);
2. genera automaticamente la documentazione interattiva OpenAPI/Swagger all'indirizzo `http://localhost:8000/docs`, molto comoda per testare le API anche senza passare dal browser della dashboard;
3. tramite `StaticFiles` consente di servire direttamente nello stesso processo `uvicorn` sia le rotte JSON sia le cartelle statiche `/dashboard` e `/presentazione`, evitando di dover configurare un terzo container con un web server separato come Nginx.

A livello di gestione delle risorse, il backend mantiene una variabile globale `_client: Optional[MongoClient] = None` inizializzata al primo accesso dalla funzione `get_db()`. In questo modo tutte le richieste HTTP condividono lo stesso pool di connessioni TCP di PyMongo verso `vitaldb_mongo`, senza dover aprire e chiudere un nuovo socket a ogni chiamata. Inoltre, prima di restituire i dati al client, il backend sostituisce sistematicamente i valori `np.nan` di Pandas con `None` (che in JSON diventa `null`), perché il formato JSON standard non ammette il token `NaN` e i browser bloccherebbero la decodifica della risposta.
Le rotte esposte da `api/main.py` sono organizzate come segue:

- `GET /health`: esegue un `ping` su MongoDB e restituisce lo stato del servizio insieme ai conteggi totali dei documenti presenti in `registry`, `vital_signals` e `anomalies_detected`;
- `GET /cases`: recupera la lista dei casi certificati chiamando `list_loaded_cases(db)` (che filtra solo `{"status": "committed"}`) e vi unisce reparto (`department`), età (`age`) e sesso (`sex`) tramite una singola pipeline di aggregazione `$group` con operatore `$first` su `vital_signals`. Come visto nel Capitolo 3, questo evita il problema delle $N+1$ query riducendo il caricamento dei 500 casi a 2 sole interrogazioni al database;
- `GET /cases/{case_id}/series`: restituisce la serie temporale del paziente richiesto. Se il client non passa il parametro opzionale `window_seconds`, l'endpoint chiama `get_case_series(db, case_id)` e restituisce il tracciato completo a 1 Hz; se invece viene specificata una finestra (ad esempio `30`, `60` o `300` secondi), chiama `downsample_case(db, case_id, window_seconds)` che delega a MongoDB il calcolo delle medie temporali con `$dateTrunc`, riducendo drasticamente il peso del JSON inviato al browser;
- `POST /cases/{case_id}/detect`: avvia l'esecuzione della pipeline ibrida di rilevamento anomalie sul caso richiesto chiamando `run_detection_pipeline(db, [case_id])`, salva i risultati in `anomalies_detected` con il pattern *Insert-Before-Delete* e restituisce al frontend il conteggio per metodo (`summary_by_method`) e l'elenco dettagliato dei secondi anomali. L'implementazione reale dell'endpoint è riportata nel Listato 5.1;
- `GET /quality`, `GET /benchmark` e `GET /evaluation`: leggono da disco e restituiscono in JSON i tre report generati dalla pipeline (`data/quality_report.json`, `data/benchmark_report.json` e

`data/ml_evaluation.json`), permettendo alla dashboard di mostrare i numeri reali dell'ultimo run eseguito.

```python
@app.post("/cases/{case_id}/detect")
def detect_case(case_id: int):
    """Esegue l'algoritmo multilivello di Anomaly Detection sul caso specificato."""
    db = get_db()
    df = run_detection_pipeline(db, [case_id])

    if df.empty:
        raise HTTPException(status_code=404, detail=f"Nessun dato temporale disponibile per case_id={case_id}")

    # Individua le colonne marcate come anomalia
    flag_cols = [c for c in df.columns if c.endswith("_anomaly")]
    anomalous_rows = df[df[flag_cols].any(axis=1)]

    # Calcola il conteggio dettagliato per ciascuna metodologia di detection
    summary_by_method = {}
    for col in flag_cols:
        method_name = col.replace("_anomaly", "")
        summary_by_method[method_name] = int(df[col].sum())

    anomalies = []
    for _, row in anomalous_rows.iterrows():
        methods = [c.replace("_anomaly", "") for c in flag_cols if bool(row[c])]

        # Converte i valori Float di Pandas sanitizzando eventuali NaN
        hr_val = float(row["Solar8000_HR"]) if "Solar8000_HR" in row and pd.notna(row["Solar8000_HR"]) else None
        spo2_val = float(row["Solar8000_PLETH_SPO2"]) if "Solar8000_PLETH_SPO2" in row and pd.notna(row["Solar8000_PLETH_SPO2"]) else None
        sbp_val = float(row["Solar8000_NIBP_SBP"]) if "Solar8000_NIBP_SBP" in row and pd.notna(row["Solar8000_NIBP_SBP"]) else None
        dbp_val = float(row["Solar8000_NIBP_DBP"]) if "Solar8000_NIBP_DBP" in row and pd.notna(row["Solar8000_NIBP_DBP"]) else None
        mbp_val = float(row["Solar8000_NIBP_MBP"]) if "Solar8000_NIBP_MBP" in row and pd.notna(row["Solar8000_NIBP_MBP"]) else None
        si_val = float(row["shock_index"]) if "shock_index" in row and pd.notna(row["shock_index"]) else None

        anomalies.append({
            "timestamp": str(row["timestamp"]),
            "methods": methods,
            "hr": hr_val, "spo2": spo2_val,
            "sbp": sbp_val, "dbp": dbp_val, "mbp": mbp_val,
            "shock_index": round(si_val, 3) if si_val is not None else None
        })

    return {
```

continua dalla pagina precedente...

```
41          "case_id": case_id,
42          "anomaly_count": len(anomalies),
43          "summary_by_method": summary_by_method,
44          "anomalies": anomalies,
45      }
```

Listato 5.1: Implementazione reale dell'endpoint `POST /cases/{case_id}/detect` in `api/main.py`.

Limiti del backend REST API

Nell'implementazione di `api/main.py` abbiamo individuato alcuni limiti che è opportuno segnalare:

1. **Ricalcolo single-case su `POST /cases/{case_id}/detect` e assenza di un endpoint `GET` di sola lettura:** non abbiamo previsto una rotta `GET /cases/{case_id}/anomalies` che si limiti a leggere da `anomalies_detected` i risultati già calcolati dal batch sui 500 casi. Quando l'utente avvia il rilevamento dalla dashboard, l'endpoint `POST` riesegue `run_detection_pipeline(db, [case_id])`, riaddestrando Isolation Forest e LSTM Autoencoder sul solo paziente selezionato e sovrascrivendo nel database le anomalie batch di quel caso;
2. **Assenza di modelli `response_model` Pydantic:** le rotte restituiscono direttamente dizionari Python sanitizzati; di conseguenza la pagina `/docs` mostra gli endpoint e i parametri di input, ma non espone lo schema tipizzato dei campi di risposta;
3. **Dettagli minori di configurazione:** l'istanza `FastAPI` dichiara `version="1.0.0"` (mentre `config.PIPELINE_VERSION` è `"1.1.0"`), l'endpoint `GET /health` conta i documenti di `registry` con `count_documents({})` senza filtrare `{"status": "committed"}`, la rotta radice `/` restituisce `404` (poiché la dashboard è montata su `/dashboard`) e il middleware CORS è lasciato aperto con `allow_origins=["*"]` per semplificare lo sviluppo locale.

5.2 Dashboard web e interfaccia utente interattiva

Per visualizzare i tracciati dei pazienti, le anomalie rilevate e le metriche dei tre layer abbiamo realizzato una dashboard web moderna accessibile all'indirizzo `http://localhost:8000/dashboard`.

Scelta tecnologica e architettura modulare del frontend. Dal punto di vista tecnologico abbiamo scelto di costruire il frontend in **HTML5, CSS3 e JavaScript puro (Vanilla JS)**, usando la libreria **Chart.js** per i grafici, senza introdurre framework come React, Vue o Angular e senza bundler come Webpack o Vite. La motivazione di questa scelta è molto concreta: per una dashboard con due sole viste principali, un framework SPA avrebbe richiesto una catena di compilazione Node.js (`npm build`) dentro Docker, appesantendo il progetto per ottenere funzioni che potevamo gestire in modo molto più diretto con JavaScript standard.
Per non scrivere comunque tutto l'HTML in un unico file gigante difficile da mantenere, abbiamo creato un piccolo caricatore modulare in `dashboard/js/loader.js`. Il file principale `dashboard/index.html` contiene dei tag segnaposto con l'attributo `data-include` (ad esempio `<div data-include="partials/sidebar.html"></div>`). All'avvio della pagina, la funzione `loadPartials()` cerca tutti i nodi `[data-include]`, scarica in parallelo i frammenti HTML tramite `Promise.all` e `fetch` aggiungendo il parametro di versione `?v=3.9` per evitare problemi di cache del browser (*cache busting*), e infine inizializza gli script della dashboard.

L'interfaccia è divisa in due viste selezionabili dalla barra superiore: la vista clinica (*Clinical Telemetry & Detection*) e la vista architetturale (*Data Lakehouse & Benchmarks*).

5.2.1 Vista di telemetria clinica e anomaly detection interattiva

La vista clinica (`dashboard/js/app.js`) è stata progettata per consentire al personale medico di esplorare in tempo reale qualsiasi dei 500 casi della coorte, valutando il consenso tra regole cliniche deterministiche e modelli non supervisionati di Machine Learning. Nelle Figure 5.1, 5.2 e 5.3 sono riportate le schermate reali dell'interfaccia aggiornata sul paziente intraoperatorio #62.

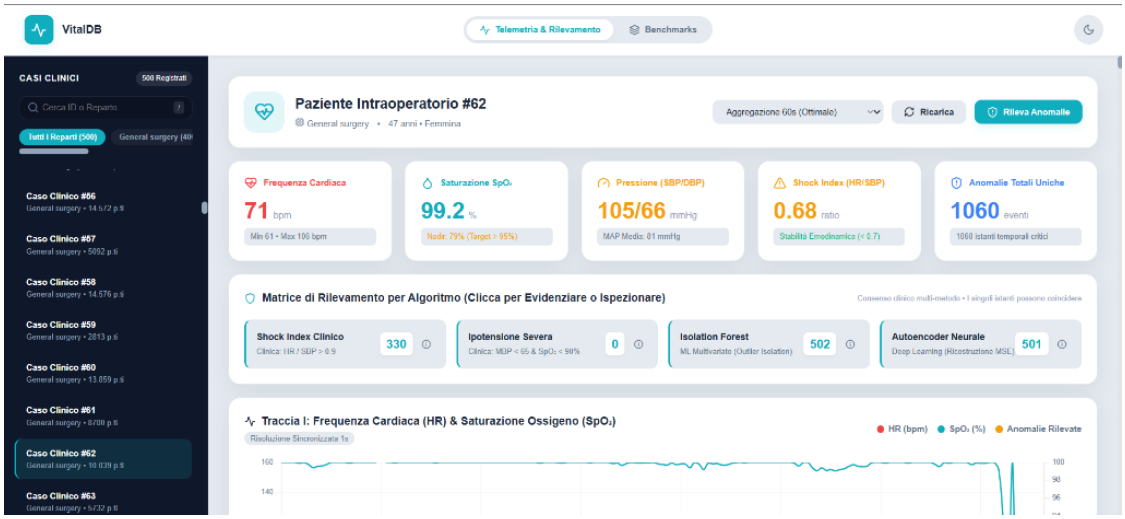


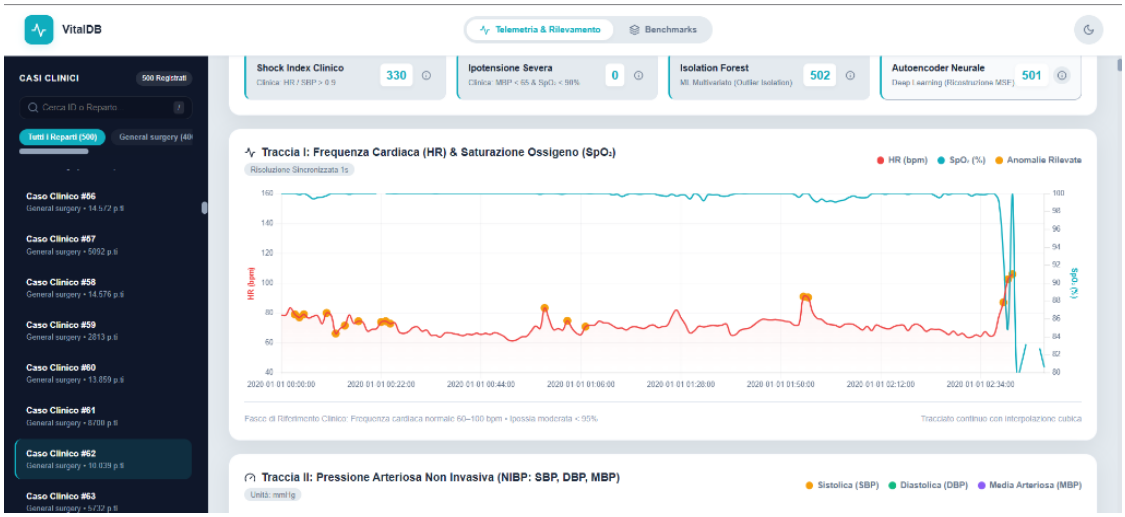
Figura 5.1: Patient HUD e Consensus Matrix per l'ispezione clinica (Caso #62, coorte di 500 casi registrati).

Come mostrato nella Figura 5.1, la parte superiore della schermata clinica comprende tre elementi chiave:

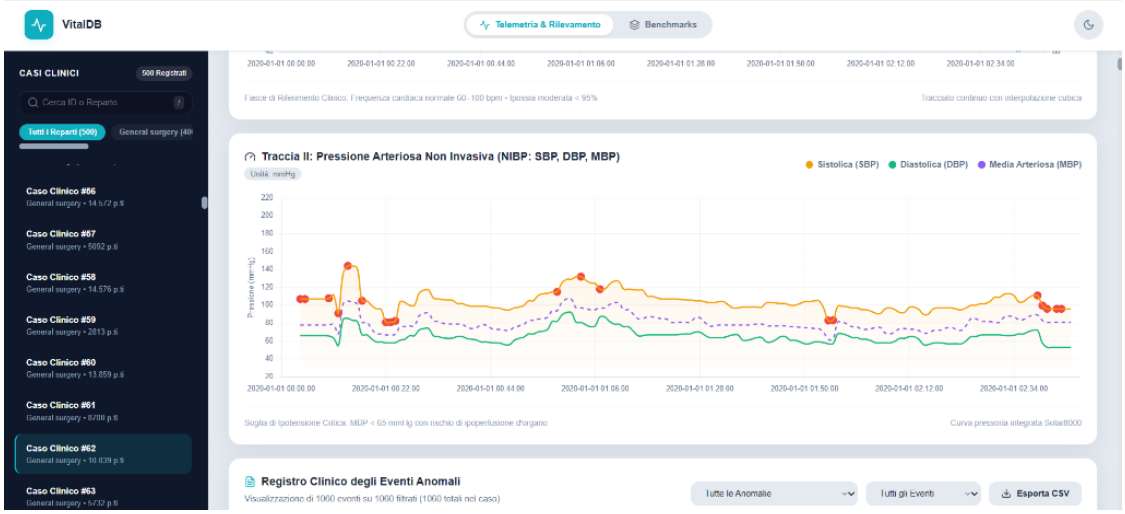
1. **Sidebar dei pazienti sincronizzata:** elenca i 500 casi `committed` con contatore dinamico (*500 Registrati*), ricerca testuale rapida per identificativo o reparto e filtri rapidi (*Tutti i Reparti*, *General surgery*, ecc.). Ciascuna voce evidenzia reparto e numero di secondi registrati nel layer Gold (es. 10.039 p.ti per il caso #62);
2. **Patient HUD e Vitals Grid:** riassume i metadati anagrafici (età 47, femmina, reparto chirurgia generale), il selettore di aggregazione temporale (`Raw 1s`, `30s`, `60s`, `5m`), il pulsante per rilanciare la detection on-demand e 5 schede di telemetria con i valori medi riassuntivi: frequenza cardiaca (71 bpm), saturazione ossigeno ($99,2\%$), pressione non invasiva ($105/66$ mmHg, MAP media 81 mmHg), Shock Index emodinamico ($0,68$, indice di stabilità) e il conteggio di **1.060 secondi anomali totali unici**;
3. **Consensus Matrix interattiva a 4 metodi:** espone il conteggio delle anomalie scoperte da ciascun approccio sul paziente selezionato: Shock Index Clinico (330), Ipotensione Severa (0), Isolation Forest multivariato (502) e Autoencoder Neurale profondo (501). Ogni scheda è cliccabile per evidenziare sul tracciato solo gli eventi di quel metodo o per aprire il pannello di audit metodologico.

Grafici temporali sincronizzati e sovrapposizione degli allarmi. I grafici temporali mostrati nella Figura 5.2 sono renderizzati tramite Chart.js e sincronizzati sull'asse del tempo in secondi di intervento:

- la **Traccia I** (Figura 5.2a) traccia simultaneamente frequenza cardiaca (linea rossa, scala 40–160 bpm) e SpO_2 (linea azzurra, scala 80–100%). I punti anomali vengono evidenziati



(a) Traccia I: Frequenza Cardiaca (HR) e Saturazione Ossigeno (SpO_2) con marker di anomalia.



(b) Traccia II: Pressione Arteriosa Sistolica, Diastolica e Media (SBP, DBP, MBP).

Figura 5.2: Tracciati temporali sincronizzati Chart.js a risoluzione di 1 secondo con sovrapposizione visiva degli eventi anomali rilevati.

con marker circolari dorati posizionati esattamente all'istante dell'evento, consentendo di identificare sia picchi di frequenza sia cadute acute di saturazione;

- la **Traccia II** (Figura 5.2b) visualizza la terna pressoria con interpolazione cubica e tratteggio sulla pressione arteriosa media (MBP, linea viola tratteggiata), confrontandola con la soglia di rischio clinico MBP < 65 mmHg per la diagnosi tempestiva di ipoperfusione d'organo.

Registro Clinico degli Eventi Anomali ed esportazione dati. Nella parte inferiore della vista telemetrica (Figura 5.3), la dashboard presenta il **Registro Clinico degli Eventi Anomali**. La tabella visualizza l'elenco cronologico di tutti i 1.060 istanti temporali anomali identificati, riportando per ciascuno:

- il timestamp assoluto UTC (`2020-01-01 00:00:53`, ecc.);
- il livello di severità semaforico (*Advisory* a 1 algoritmo, *Warning* a 2 algoritmi concordi, *Critical* a 3 o 4 algoritmi);

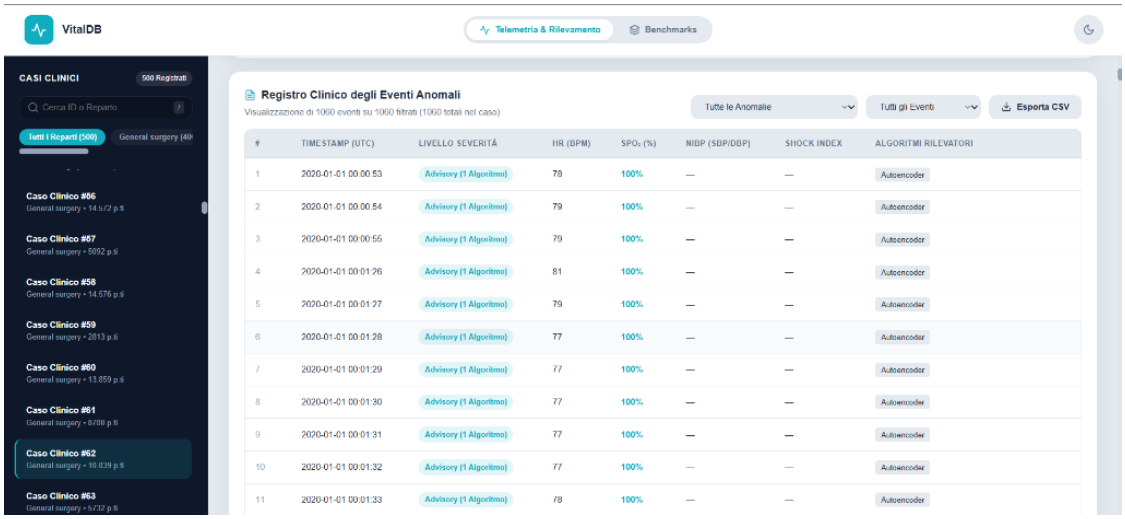


Figura 5.3: Registro clinico degli eventi anomali con severità, valori fisiologici istantanei ed esportazione CSV.

- i valori istantanei dei parametri fisiologici al momento dell'allarme (HR, SpO_2, SBP/DBP, Shock Index);
- l'elenco esplicito dei metodi che hanno votato per l'anomalia (es. *Autoencoder*, *Isolation Forest*, *Shock Index*);
- il pulsante **Esporta CSV** per scaricare l'intero log degli eventi da utilizzare per verifiche cliniche retrospettive o archiviazione medico-legale.

5.2.2 Vista di Data Lakehouse, Governance e Benchmark

La seconda vista (`dashboard/js/lakehouse.js` e `partials/lakehouse-view.html`) presenta i risultati tecnici della pipeline leggendo dinamicamente i dati da `/benchmark`, `/evaluation` e `/quality`:

- **Confronto di Storage e Query Latency:** mostra l'occupazione dei tre layer (**33,95 MB** in Bronze, **44,52 MB** in Silver con badge `+31.1% con flag DQ`, e **15,60 MB** in Gold con **-54,1%** rispetto a Bronze e **-65,0%** rispetto a Silver, più la nota di **+2,56 MB** per gli indici B-Tree) e il tempo medio di query (**12,02 ms** su MongoDB contro **26,95 ms** da Parquet, pari a **2,2**×). Durante l'ultima revisione abbiamo corretto il calcolo della larghezza delle barre grafiche usando `Math.max(bronzeMB, silverMB, goldMB)` come riferimento (perché essendo Silver più grande di Bronze, rapportare Silver a Bronze produceva una barra oltre il 100%) e abbiamo aggiornato i testi descrittivi del diagramma che in precedenza contenevano numeri statici di una vecchia prova su 50 casi;
- **Tabella di valutazione ML:** mostra i valori di Precision, Recall, F1-score e **PR-AUC (Average Precision)** di Isolation Forest e LSTM Autoencoder sui **5.402.386 punti** della coorte Gold;
- **Tabella del Data Quality Report:** mostra il riepilogo globale dei campioni (`5.565.384` righe Bronze, `5.402.386` righe Silver/Gold, `162.998` righe vuote rimosse pari al `2,93%`) e il dettaglio per paziente di valori mancanti e outlier.

Limiti della dashboard web

Anche sul fronte dell'interfaccia grafica segnaliamo con trasparenza tre limiti attuali:

1. **Mancata esposizione visiva dei metadati di lineage:** sebbene gli endpoint `/quality`, `/benchmark` ed `/evaluation` restituiscano nei rispettivi JSON i campi `run_id`, `pipeline_version` e `generated_at`, la dashboard non li mostra in nessun badge o

intestazione della pagina;
2. **Visualizzazione parziale della tabella Data Quality:** per non appesantire il DOM del browser, in `dashboard/js/lakehouse.js` la tabella dei casi mostra solo i primi 15 pazienti (`data.cases_detail.slice(0, 15)`) su 500 e non espone le metriche dettagliate per singolo sensore (`sensor_metrics`) che pure sono salvate nel `quality_report.json` e nel `registry`;
3. **Valori di fallback statici nel codice client:** nel template HTML e nel file `lakehouse.js` sono rimasti alcuni valori numerici di fallback che verrebbero mostrati qualora il backend rispondesse con errore `404` o `500` (ad esempio se i report JSON non fossero ancora stati generati).

5.3 Containerizzazione Docker, orchestrazione, test (`pytest`) e reset

Per rendere il progetto eseguibile allo stesso modo su qualsiasi macchina (Windows, macOS o Linux) senza dover installare manualmente MongoDB 8.0 o configurare ambienti Python locali, abbiamo incapsulato l'intero sistema in due container gestiti da `docker-compose.yml`, come rappresentato nella Figura 5.4.

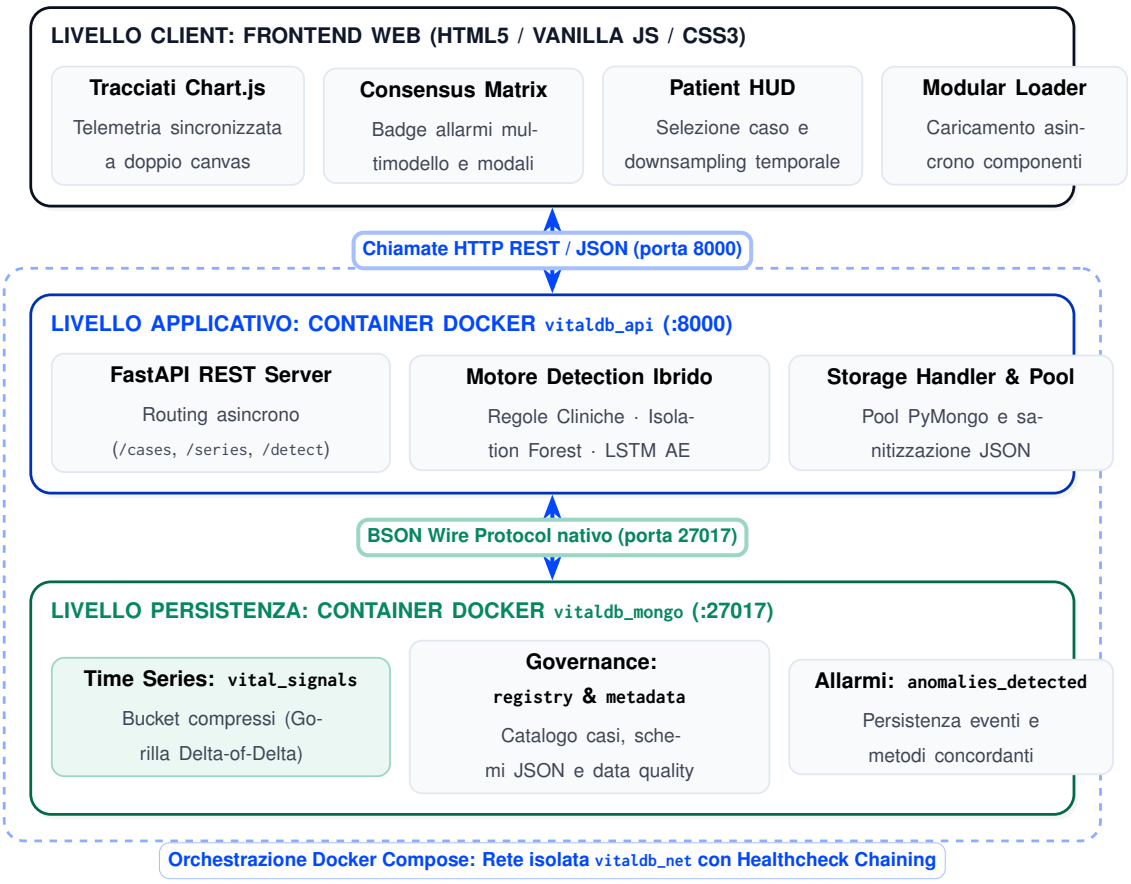


Figura 5.4: Architettura multi-livello dell'applicazione e containerizzazione Docker Compose.

I due servizi collegati sulla rete virtuale `vitaldb_net` sono:

1. `vitaldb_mongo` (immagine `mongo:8.0`): avvia il database con l'opzione `--replSet rs0`, monta il volume persistente `mongo_data` e gli script di inizializzazione in `/docker-entrypoint-initdb.d/`. Poiché MongoDB richiede alcuni secondi per inizializzare il

Replica Set all'avvio, abbiamo configurato un `healthcheck` basato su `mongosh` che controlla `rs.status()` (o lancia `rs.initiate()` al primo avvio);

2. `vitaldb_api` (costruito dal `Dockerfile` su base `python:3.12-slim`): installa le dipendenze di `requirements.txt` (inclusa la build CPU di PyTorch), monta le cartelle del progetto e avvia `uvicorn api.main:app --host 0.0.0.0 --port 8000`. Grazie alla clausola `depends_on: {mongo: {condition: service_healthy}}`, il container dell'API attende che il Replica Set di MongoDB sia effettivamente pronto prima di partire.

5.3.1 Orchestrazione della pipeline end-to-end (`scripts/run_pipeline.py`)

Nelle prime fasi del progetto, l'avvio della pipeline sui 500 casi richiedeva il lancio manuale in sequenza di 6 comandi separati dentro il container `vitaldb_api`:

1. `python src/bronze/download.py --max-cases 500` (genera il nuovo `run_id` e scarica il layer Bronze);
2. `python src/silver/clean.py` (pulisce i dati, genera il layer Silver e `data/quality_report.json`);
3. `python src/gold/load_mongo.py` (carica `vital_signals` e `registry` con protocollo a due fasi);
4. `python src/detection/detector.py` (esegue regole cliniche + ML e aggiorna `anomalies_detected`);
5. `python src/analysis/benchmark_etl.py` (aggiorna `data/benchmark_report.json`);
6. `python src/analysis/evaluate_models.py` (aggiorna `data/ml_evaluation.json`).

Un **limite operativo** che avevamo riscontrato lanciando i singoli comandi da terminale era il rischio di disallineamento dello stato qualora l'esecuzione fosse interrotta prematuramente o eseguita con perimetri eterogenei (ad esempio Bronze scaricato su 500 casi ma Gold rimasto fermo a 50). Per superare questa criticità e garantire riproducibilità deterministica, abbiamo sviluppato lo script orchestratore `scripts/run_pipeline.py`, eseguibile direttamente via Docker:

```
docker exec vitaldb_api python -u scripts/run_pipeline.py
```

Lo script incapsula l'intera catena applicando una politica **fail-fast**: esegue i 6 step in sequenza ordinata, intercetta il codice di uscita di ciascun sottoprocesso e blocca immediatamente l'esecuzione se uno step fallisce (`exit code != 0`), stampando per ciascuna fase i tempi parziali e la durata complessiva dell'elaborazione.

5.3.2 Suite di 38 test automatizzati (`pytest`) e script di reset

Per verificare che le modifiche introdotte durante lo sviluppo non rompessero la pulizia dei dati, la governance o le API, abbiamo scritto una suite di **38 test unitari** eseguibili con `pytest` e suddivisi nei **6 moduli reali** presenti nella cartella `tests/`:

- `tests/test_config.py` (**2 test**): verifica che tutte le tracce in `TRACK_SPECS` abbiano limiti fisiologici validi ($\min < \max$) e che le costanti cliniche siano coerenti (in particolare che la soglia di freschezza clinica `NIBP_MAX_CLINICAL_AGE_S = 120s` non superi mai il limite di *forward-fill* `NIBP_STALENESS_LIMIT_S = 300s`);
- `tests/test_bronze.py` (**3 test**): verifica la scrittura atomica su file temporaneo (`.tmp` $\rightarrow$ `.parquet`), il salvataggio delle tracce del singolo paziente e la generazione di `download_manifest.json` con `run_id`;
- `tests/test_etl.py` (**4 test**): collauda le funzioni di `src/silver/clean.py`, verificando che `interpolate_short_gaps` colmi solo i buchi interni ≤ 5 s lasciando `NaN` le code finali prive di campioni successivi, che `flag_outliers` individui gli artefatti prima dell'imputazione, che i 5 flag `*_imputed` siano tracciati separatamente per canale e che per ogni traccia valga sempre l'identità `observed` + `imputed` + `missing` = `rows_cleaned`;
- `tests/test_gold.py` (**16 test**): verifica la conversione `dataframe_to_timeseries_docs` (omissione dei campi `NaN` e preservazione di `nibp_age_s` e dei 5 flag `*_imputed`), il quality gate `validate_records`, la struttura del documento `registry` con `sensor_metrics` e `run_id`,

il filtro `{"status": "committed"}` in `list_loaded_cases` e la funzione di auto-guarigione `recover_incomplete_loads`;
- `tests/test_detection.py` (**9 test**): verifica il calcolo di Shock Index e ipotensione severa, il silenziamento degli allarmi quando `nibp_age_s > 120s`, l'isolamento delle finestre scorrevoli tra pazienti diversi nell'`LSTMAutoencoderDetector`, la presenza del proiettore `nn.Linear` e l'assenza di *data leakage* nel `.bfill()` raggruppato (`test_imputation_no_bfill_leakage_across_cases`);
- `tests/test_api.py` (**4 test**): testa tramite `TestClient` le rotte `/health`, `/cases`, `/cases/{id}/series` e `POST /cases/{id}/detect`. In particolare, in `test_api_detect_endpoint` abbiamo usato `monkeypatch.setattr(detector_mod, "save_anomalies_to_mongo", lambda db, df: len(df))` per evitare che l'esecuzione dei test sovrascrivesse nel database le anomalie reali calcolate sui 500 casi.

Infine, per poter ripartire da zero in modo pulito quando necessario, abbiamo predisposto lo script `scripts/reset_pipeline.py` (eseguibile con il flag di conferma `-yes` / `-y`). Lo script svuota le tre collezioni MongoDB (`vital_signals`, `registry`, `anomalies_detected`), cancella i file Parquet dentro `data/bronze/` e `data/silver/` e rimuove i 4 file JSON di report insieme a `data/.current_run_id`.

5.4 Conclusioni, riepilogo dei limiti per layer e sviluppi futuri

Il progetto realizzato ci ha permesso di affrontare dall'inizio alla fine tutte le fasi di costruzione di una pipeline dati e di un sistema di **anomaly detection** su serie storiche cliniche reali, lavorando su una coorte di **500 interventi chirurgici** del dataset VitalDB (**5.565.384 righe grezze** in Bronze e **5.402.386 secondi bonificati** in Silver e Gold).
Dall'esperienza pratica di sviluppo possiamo trarre tre conclusioni principali:

1. **Separare i ruoli dei tre layer (Medallion Architecture) semplifica sia il debug sia le prestazioni:** mantenere il layer **Bronze** immutabile in Parquet (**33,95 MB**) ci ha evitato di riscaricare i dati da rete a ogni modifica del codice; usare il layer **Silver** (**44,52 MB**, $+31,1\%$) per arricchire i dati con 12 colonne di qualità (`nibp_age_s`, 5 flag `*_outlier` e 5 flag `*_imputed`) senza cancellare gli outlier ci ha permesso di conservare la tracciabilità completa; infine, delegare al layer **Gold** su MongoDB 8.0 Time Series il solo dato utile per le query ha ridotto lo spazio fisico dei dati a **15,60 MB** ($-65,0\%$ rispetto a Silver, più **2,56 MB** di indici B-Tree) e ha dimezzato il tempo medio di lettura sul singolo paziente (**12,02 ms** contro **26,95 ms**);
2. **La qualità del dato condiziona direttamente gli algoritmi a valle:** scelte apparentemente piccole — come rilevare gli outlier prima del *forward-fill*, limitare la propagazione NIBP a 300 secondi in Silver ma stringerla a `nibp_age_s` ≤ 120 s per le regole cliniche, o raggruppare correttamente per `case_id` anche il `.bfill()` — fanno la differenza tra un sistema che genera falsi allarmi continui e uno metodologicamente corretto;
3. **Regole cliniche e modelli non supervisionati svolgono compiti diversi ma complementari:** le regole deterministiche danno allarmi immediati e spiegabili su soglie note (SI $> 0,9$ e MBP $< 65 \wedge \text{SpO}_2 < 90\%$), mentre Isolation Forest (PR-AUC $0,105$, $\approx 2,2\times$ il caso) e l'LSTM Autoencoder (PR-AUC $0,067$, $\approx 1,4\times$ il caso) fungono da motori di ranking multivariato sulle 8 feature dinamiche, segnalando anche instabilità del ritmo o cadute rapide sub-soglia.

5.4.1 Quadro riepilogativo dei limiti riscontrati lungo l'architettura

Per offrire una visione d'insieme chiara e onesta del progetto, riassumiamo qui tutti i limiti strutturali e implementativi che abbiamo analizzato nei singoli capitoli, suddivisi per componente:

- **Layer Bronze (`src/bronze/download.py`):**

1. il controllo di idempotenza sui file già presenti (`skipped_existing`) verifica solo il passo tra i primi due campioni (`Time.iloc[1] - Time.iloc[0]`) e la presenza delle 5 colonne, senza ri-verificare l'integrità dell'intero file;
2. la tabella `lab_data.parquet` viene scaricata con *graceful fallback* ma non viene mai utilizzata nei layer successivi;
3. il file `download_manifest.json` registra il conteggio aggregato dei casi ma non l'elenco o gli hash MD5/SHA256 dei singoli file Parquet.

- **Layer Silver (`src/silver/clean.py`):**
  1. i valori marcati come `*_outlier` non vengono rimossi prima dell'imputazione in Silver: un outlier sul bracciale NIBP può quindi essere propagato in avanti dal *forward-fill* fino a 300 secondi nel file Parquet Silver (problema che abbiamo dovuto neutralizzare a valle mascherando gli outlier a `NaN` in `load_from_gold`);
  2. l'interpolazione lineare (≤ 5 s) viene eseguita prima di calcolare la maschera `was_na`, perciò i buchi brevi interpolati vengono contati nel report come `observed` anziché come `imputed`;
  3. non esiste una soglia di scarto per caso (*case-level quality gate*): anche un paziente con lunghi tratti mancanti viene comunque salvato in Silver e passato a Gold.
- **Layer Gold, Data Lakehouse "simulato" e Benchmark (`src/gold/load_mongo.py`):**
  1. poiché le Time Series Collections di MongoDB 8.0 non supportano `$jsonSchema` né transazioni multi-documento verso il `registry`, le garanzie di schema e di atomicità (*Two-Phase Load*) sono imposte dagli script Python e non dal motore del database;
  2. la lineage del `run_id` si ferma al `registry` e ai file JSON, ma non è scritta dentro i singoli documenti di `vital_signals`;
  3. il timestamp usa una data base fittizia (`2020-01-01 + Time`) perché VitalDB fornisce solo i secondi relativi dall'inizio dell'intervento;
  4. nel benchmark (`benchmark_etl.py`) il risparmio di storage (`-54,1%` vs Bronze, `-65,0%` vs Silver) è calcolato su `gold_storage_mb = 15,60 MB` al netto dei `2,56 MB` di indici B-Tree, e il confronto di latenza legge da Parquet tutte le 18 colonne contro le 14 proiettate da MongoDB.
- **Anomaly Detection e valutazione ML (`src/detection/detector.py`, `evaluate_models.py`):**
  1. fissare `contamination=0.05` e il 95° percentile impone per costruzione che i due modelli ML selezionino sempre circa il 5% dei punti valutati (agendo come meccanismo di ranking e non come stima della prevalenza reale);
  2. la valutazione si basa su una *pseudo-ground truth* derivata dalle stesse tracce tramite le regole cliniche, misurando quindi la concordanza interna e non la performance su diagnosi mediche indipendenti;
  3. i modelli non vengono salvati su disco (`.pkl`/`.pt`) e vengono riaddestrati a ogni esecuzione;
  4. i documenti di `anomalies_detected` non contengono un `detection_run_id` che identifichi la versione dei modelli e gli iperparametri usati.
- **Backend API, Dashboard e Orchestrazione (`api/main.py`, `dashboard/`, `scripts/`):**
  1. l'endpoint `POST /cases/{id}/detect` riaddestra i modelli sul singolo paziente selezionato e sovrascrive in `anomalies_detected` le anomalie calcolate dal batch sui 500 casi, in assenza di un endpoint `GET /cases/{id}/anomalies` di sola lettura;
  2. la dashboard non visualizza a schermo `run_id` e `pipeline_version`, mostra solo i primi 15 casi nella tabella Data Quality e mantiene alcuni valori di fallback cablati nel codice client;
  3. l'orchestrazione tramite `run_pipeline.py` è sincrona e monoprocesso nel container, priva di un motore distribuito a grafi di dipendenze (DAG) con code asincrone e retry automatici.

5.4.2 Sviluppi futuri

A partire dai limiti appena descritti, le estensioni più naturali per portare il progetto a un livello successivo sono quattro:

- **Salvataggio dei modelli addestrati e separazione tra lettura e ricalcolo nell'API:** serializzare su disco lo `StandardScaler`, l'istanza di `IsolationForest` e i pesi PyTorch (`.pt`) dell'Autoencoder addestrati sui 500 casi, introducendo una rotta `GET /cases/{id}/anomalies` per la consultazione immediata dalla dashboard senza riaddestrare i modelli sul singolo paziente;
- **Doppia lineage granulare (`pipeline_run_id` e `detection_run_id`):** inserire direttamente nei documenti di `vital_signals` e `anomalies_detected` sia l'identificativo del caricamento dati sia quello dell'esecuzione dei modelli, esponendoli anche nell'intestazione della dashboard;
- **Quality gate bloccante a livello di caso:** introdurre una soglia minima di integrità del dato prima del passaggio a Gold (ad esempio escludere automaticamente dal caricamento i casi con copertura temporale del segnale cardiaco inferiore al 50%);
- **Orchestrazione distribuita a DAG e integrazione di nuove tracce VitalDB:** evolvere lo script `run_pipeline.py` in un workflow formale con Apache Airflow o Prefect per gestire esecuzioni concorrenti, code di retry ed alert via webhook, ed estendere l'analisi anche alla pressione arteriosa invasiva (`ART`), alla capnografia ($EtCO_2$) e all'indice bispettrale (`BIS`).